ЛЕКЦІЯ 6. РОБОТА З API

ЩО ТАКЕ API?

API (Application Programming Interface) — це інтерфейс програмування застосунків, який дозволяє різним програмам взаємодіяти між собою. Це набір правил і методів, за допомогою яких одна програма може звертатися до функцій або даних іншої програми.

Як працює API?

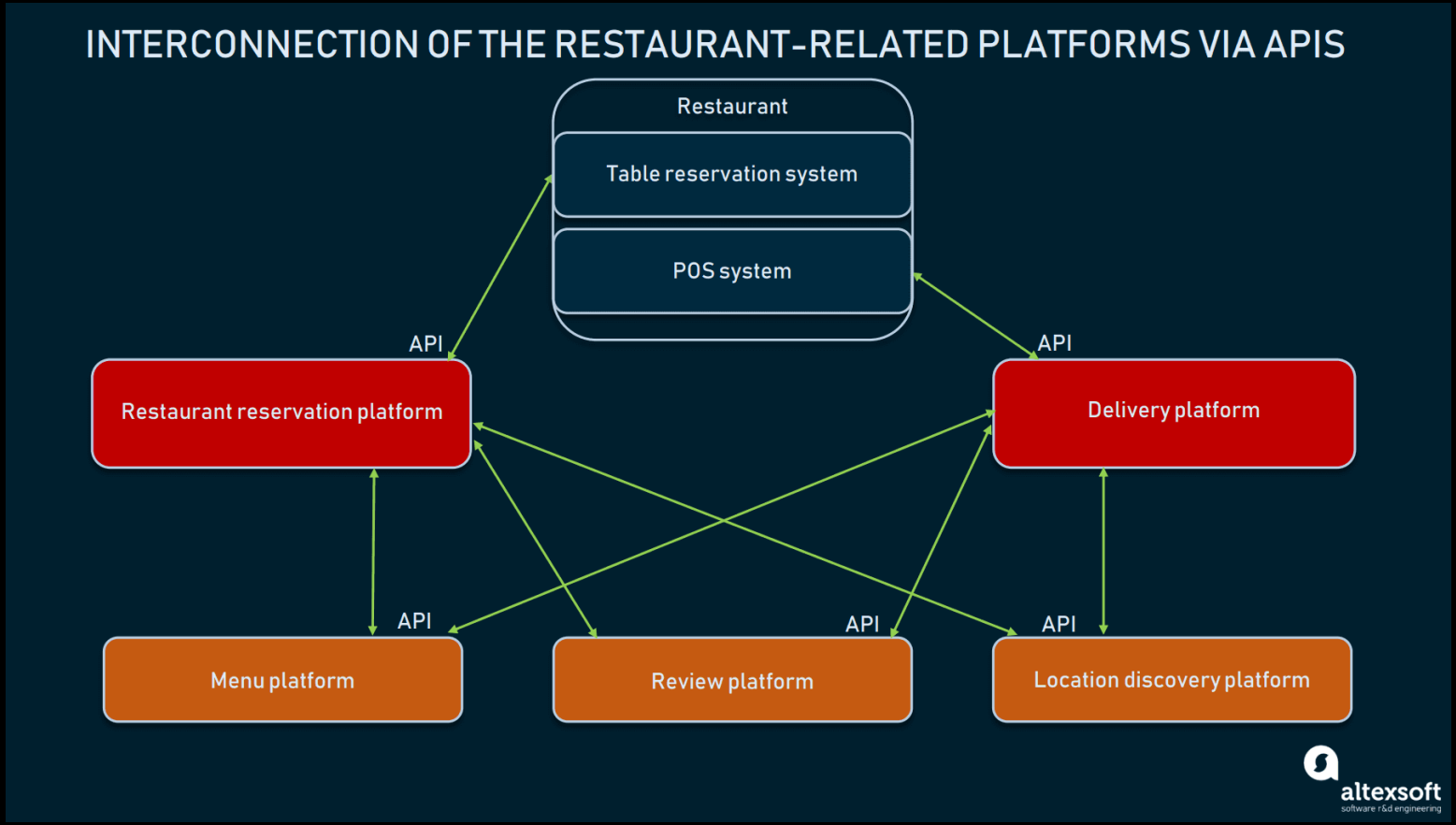
API працює як посередник між двома системами:

1. **Клієнтська програма** (наприклад, мобільний застосунок) надсилає запит через API.
2. **Сервер** обробляє цей запит і повертає відповідь у визначеному форматі (наприклад, JSON або XML).
3. **Клієнт** отримує відповідь і використовує її для відображення або подальших дій.

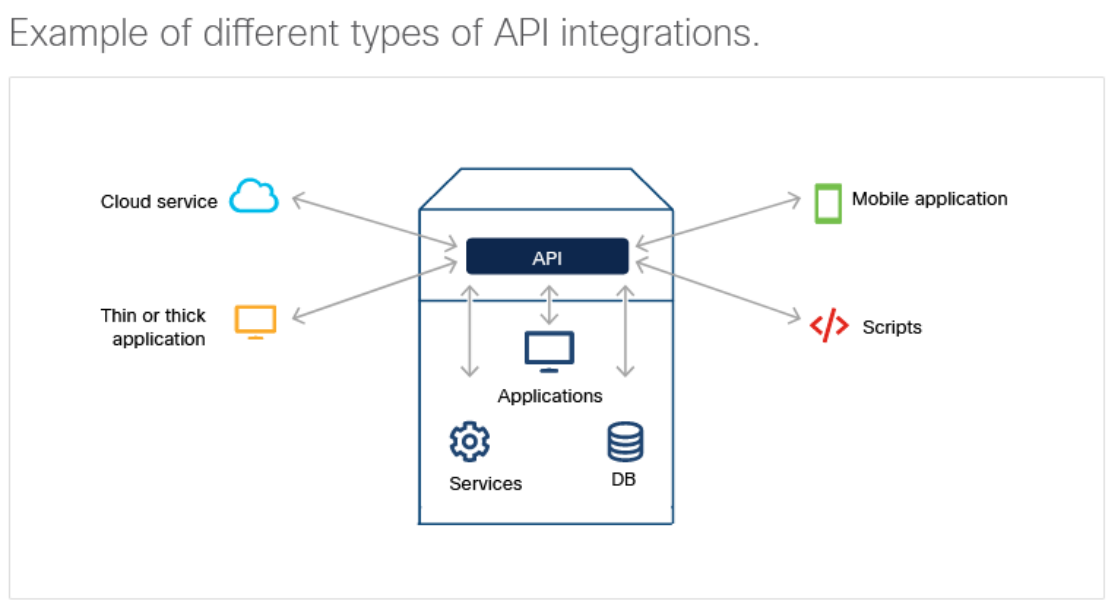
ПРИКЛАД API

Прикладом можливостей використання API є програма для рекомендацій щодо ресторанів, яка повертає список відповідних ресторанів у певному районі.

Замість того, щоб створювати функцію відображення з нуля, додаток інтегрує сторонній API, щоб забезпечити функціональність карти. Творець API визначає, як і за яких обставин розробники можуть отримати доступ до інтерфейсу.



АСОЦІАЦІЇ



На API можна дивитись, як на кнопки на приладовій панелі автомобіля. Коли ви заводите автомобіль, ви не бачите і можете навіть не здогадуватись про те, що двигун запускає електронну систему запалювання, яка змушує газ потрапляти в двигун і поршні починають рухатися. Все, що ви знаєте, це те, що якщо ви хочете, щоб автомобіль завівся.

Ви повертаєте ключ або натискаєте кнопку ON Engine. Якщо натиснути іншу кнопку, наприклад, кнопку ввімкнення радіо, автомобіль не заведеться, оскільки це не було визначенням, яке автомобіль (програма) визначив для запуску двигуна.

Автомобіль (додаток) сам по собі може робити набагато більше, ніж запускати двигун, але ці речі не доступні для водія (стороннього користувача).

НАВІЩО ПОТРІБНІ API?

API зазвичай створюються для програмного використання іншими додатками, тому їх називають інтерфейсами програмного забезпечення. Але їх не обов’язково потрібно використовувати програмно; їх також можуть використовувати люди, які хочуть взаємодіяти з програмою вручну.

Далі розглянемо декілька з багатьох варіантів використання API.

API І ЗАВДАННЯ АВТОМАТИЗАЦІЇ

Часто API використовують для створення сценаріїв, які можуть автоматично та програмно виконувати ручні завдання.

Приклад: менеджеру деякої компанії в останній день кожного періоду оплати потрібно вручну ввійти на портал і завантажити табель для всіх своїх підзвітних працівників. Потім менеджер вручну повинен підсумувати кількість годин, відпрацьованих кожною особою, щоб потім обчислити заробітну платню.

Цей трудомісткий для менеджера процес підлягає автоматизації, для цього потрібно створити автоматизований сценарій, який викликає API порталу, щоб отримати дані з кожного табеля та обчислити загальну суму на основі цих даних. Цей сценарій може роздрукувати список загальної кількості відпрацьованих годин для кожного працівника у зручному для користувача форматі.

API ТА ІНТЕГРАЦІЯ ДАНИХ

Інтеграція даних означає, що програма може використовувати дані, надані іншою програмою, або реагувати на них.

Приклад: багато веб-сайтів електронної комерції використовують платіжні послуги, доступ до яких здійснюється через REST API. Інтерфейс платежу дозволяє сайту постачальника передавати відносно малоцінні дані, такі як опис придбаного об'єкта та його ціна. Потім користувач самостійно аутентифікується у платіжній службі, щоб підтвердити платіж. Це дозволяє постачальнику отримувати оплату, не потребуючи прямого доступу до даних кредитної картки клієнта. Продавець отримує у відповідь код підтвердження для подальшого використання бухгалтерією.

API ТА ФУНКЦІОНАЛЬНІСТЬ

Під **функціональністю** розуміємо, що програма може інтегрувати функціональні можливості іншої програми у свій продукт.

Приклад: багато онлайн-сервісів, таких як Uber, Uklon, тощо обмінюються даними з Google Maps для створення й оптимізації маршрутів подорожі. Вони також вбудовують функції з Google Maps у свої власні програми та веб-сайти, щоб представити карти маршрутів у реальному часі.

ПРИЧИНИ ПОПУЛЯРНОСТІ API

API стали популярними через їхню гнучкість, зручність та важливу роль у цифровій екосистемі. Ось основні причини їхньої популярності:

1. **Автоматизація процесів.** API дозволяють автоматизувати рутинні завдання, зменшуючи ручну працю та підвищуючи ефективність. Наприклад, API CRM-системи може автоматично оновлювати дані клієнтів.
2. **Легка інтеграція.** За допомогою API різні сервіси можуть легко взаємодіяти. Наприклад, сайти електронної комерції інтегрують платіжні шлюзи (PayPal, Stripe) через API, спрощуючи оплату для користувачів.
3. **Гнучкість та масштабованість.** Розробники можуть використовувати API, щоб розширювати можливості своїх програм без необхідності створювати все з нуля. Це особливо важливо для хмарних сервісів.
4. **Можливість створення екосистем.** Великі компанії, такі як Google, Facebook чи Amazon, створюють API, щоб розробники могли інтегрувати їхні сервіси у власні продукти. Це сприяє розвитку екосистеми та залученню нових користувачів.
5. **Безпека.** API можуть мати обмежений доступ до даних, надаючи тільки необхідну інформацію та захищаючи основну систему від прямих атак. Використання токенів та аутентифікації (OAuth, API-ключі) підвищує рівень безпеки.
6. **Економія ресурсів.** Замість створення власних рішень компанії можуть використовувати готові API, що дозволяє швидше запускати продукти та скорочувати витрати на розробку.
7. **Швидкість передачі даних.** API використовують оптимізовані формати даних (наприклад, JSON або XML), що забезпечує швидку та ефективну взаємодію між сервісами.

СТИЛІ РОЗРОБКИ API

API можна надавати одним із двох способів: **синхронно** або **асинхронно**.

Різницю між ними необхідно чітко розуміти, оскільки програма, яка використовує API, керує відповіддю по-різному залежно від дизайну API. Кожен дизайн має власну мету, а також власний набір складностей на стороні клієнта та/або сервера. Набір API продукту може складатися як із синхронного, так і з асинхронного проектів, де кожен дизайн API не залежить від інших. Однак для найкращих практик логіка дизайну повинна бути послідовною.

СИНХРОННІ API

Синхронні API відповідають на запит безпосередньо, зазвичай надаючи дані (або іншу відповідь) негайно.

Коли API є синхронними?

API зазвичай розроблені так, щоб бути синхронними, коли дані для запиту легко доступні, наприклад, коли дані зберігаються в базі даних або у внутрішній пам'яті. Сервер може миттєво отримати ці дані та негайно відповісти.

Переваги синхронного дизайну API

Синхронні API дозволяють програмі отримувати дані негайно. Якщо API розроблено правильно, програма матиме кращу продуктивність, оскільки все відбувається швидко. Однак, якщо він неправильно розроблений, запит API буде слабким місцем, оскільки програма має чекати відповіді.

Обробка на стороні клієнта

Програма, яка надсилає запит API, повинна чекати відповіді, перш ніж виконувати будь-які додаткові завдання на виконання коду.

АСИНХРОННІ API

Асинхронні API не надають негайну відповідь на сам запит. Проте негайно присилають відповідь, яка означає, що запит отримано, але ця відповідь не містить жодних фактичних даних. Сервер обробляє запит, що може зайняти деякий час, і надсилає сповіщення (або запускає зворотний виклик) з даними після обробки запиту. Потім клієнт може діяти на основі отриманих даних. API зазвичай розроблені як асинхронні, коли запит є дією, яка займає деякий час для обробки сервером, або якщо дані недоступні.

Наприклад, якщо сервер повинен зробити запит до віддаленої служби для отримання даних, він не може гарантувати, що він отримає дані негайно, щоб відправити їх назад клієнту. Те, що API є асинхронним, не обов'язково означає, що клієнт не отримає дані негайно. Це лише означає, що негайна відповідь з даними не гарантується.

АСИНХРОННІ API

Переваги асинхронного дизайну API

Асинхронні API дозволяють програмі продовжувати виконання без блокування протягом часу, який потрібен серверу для обробки запиту. У результаті програма може мати кращу продуктивність, оскільки вона може виконувати багато завдань і робити інші запити. Однак непотрібне або надмірне використання асинхронних викликів може мати протилежний вплив на продуктивність.

Обробка на стороні клієнта

При асинхронній обробці дизайн API на стороні сервера визначає, що ви хочете робити на стороні клієнта. Іноді клієнт може встановити механізм прослуховування або зворотного виклику для отримання цих сповіщень і обробки їх, коли вони отримані. Залежно від дизайну програми, вашому клієнту також може знадобитися черга для зберігання запитів для підтримки замовлення для обробки. Інші конструкції API потребують, щоб клієнт мав механізм опитування, щоб дізнатися статус і перебіг даного запиту.

АРХІТЕКТУРНІ СТИЛІ API

Програма сама визначає, як треті сторони взаємодіють із нею, а це означає, що не можна виокремити якогось «стандартного» способу створення API. Однак, незважаючи на те, що технічно програма може виявити будь-який довільний інтерфейс, найкращим виходом буде слідувати стандартам, протоколам і специфічним архітектурним стилям.

Завдяки цьому споживачам API буде набагато легше вивчити і зрозуміти API, оскільки поняття вже знайомі.

Найпопулярніші типи архітектурних стилів API – це

- ✅ **REST** — для стандартних веб- та мобільних додатків
  ✅ **SOAP** — для безпечних корпоративних систем
  ✅ **GraphQL** — для гнучких запитів
  ✅ **gRPC** — для швидких мікросервісів
  ✅ **WebSockets** — для роботи в реальному часі

СТИЛЬ GRPC

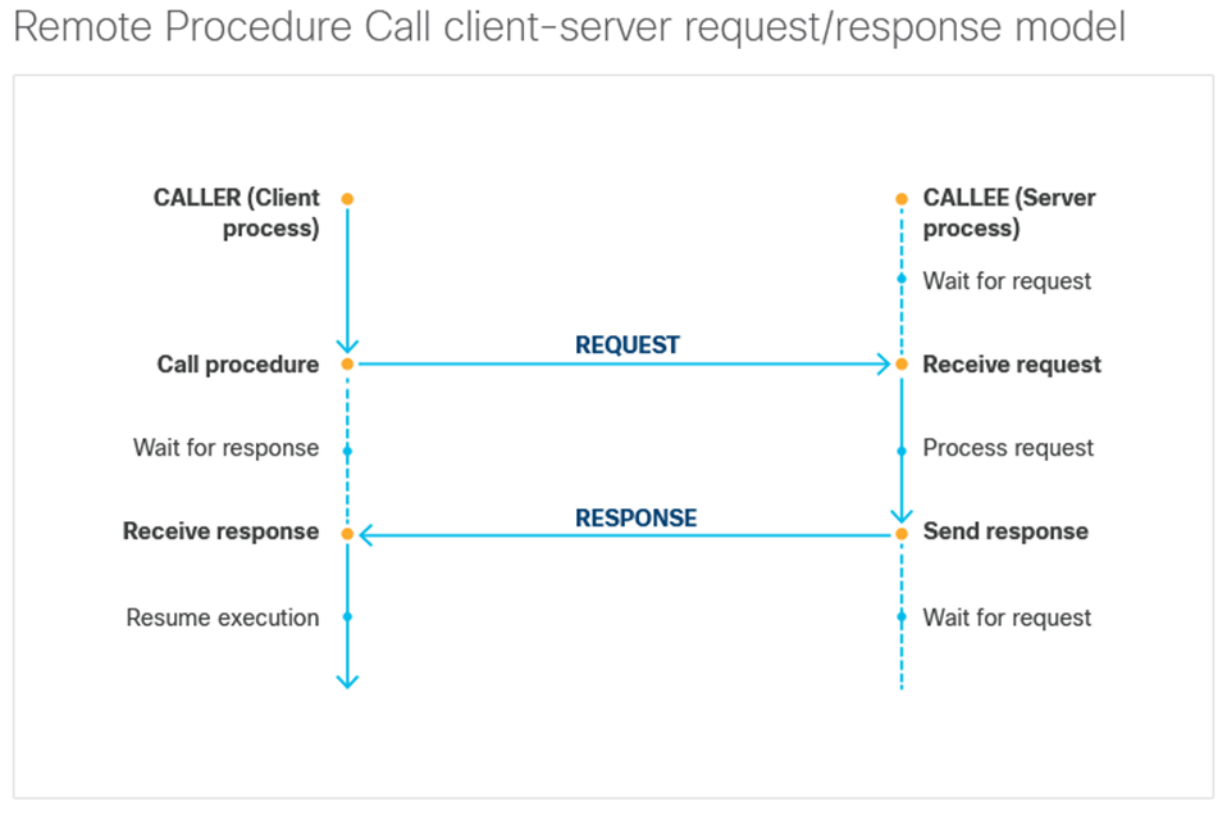
Віддалений виклик процедур Google (Google Remote Procedure Call - gRPC) — це модель запиту-відповіді, яка дозволяє додатку (діє як клієнт) здійснювати виклик процедури до іншої програми (діє як сервер). «Серверна» програма зазвичай розташована в іншій системі у мережі. При застосуванні gRPC клієнт зазвичай не знає, що запит процедури виконується віддалено, оскільки запит надходить до рівня, який приховує ці деталі. Що стосується клієнта, ці виклики процедур є просто діями, які він хоче виконати. Іншими словами, для клієнта віддалений виклик процедури є просто методом з аргументами. Коли його викликають, виконується метод і повертаються результати.

СТИЛЬ GRPC

У найбільш поширеному використанні gRPC клієнт робить синхронний запит до сервера і блокується, поки сервер обробляє запит. Коли сервер виконує запит, він надсилає відповідь клієнту, який розблокує його процес. (Це не стосується асинхронних запитів.) gRPC — це стиль API, який можна застосувати до різних транспортних протоколів.

Приклади реалізації включають:

- XML-RPC
- JSON-RPC
- NFS (Network File System - Мережева файлова система)
- Simple Object Access Protocol (SOAP)



СТИЛЬ SOAP

SOAP (Simple Object Access Protocol) – це протокол обміну повідомленнями. Він використовується для зв'язку між додатками, які можуть бути на різних платформах або створені на різних мовах програмування. Це протокол на основі XML, розроблений компанією Microsoft. SOAP зазвичай використовується з транспортним протоколом HyperText Transfer Protocol (HTTP), але його можна застосувати до інших протоколів (SMTP, TCP). SOAP є незалежним, розширюваним і нейтральним.

Незалежний. SOAP був розроблений так, щоб усі типи додатків могли взаємодіяти один з одним. Програми можуть бути створені з використанням різних мов програмування, можуть працювати на різних операційних системах і можуть бути максимально різними.

Розширюваний. Сам SOAP вважається додатком XML, тому на його основі можна створювати розширення. Ця розширюваність означає, що ви можете додати такі функції, як надійність і безпека.

Нейтральний. SOAP можна використовувати через будь-який протокол, включаючи HTTP, SMTP, TCP, UDP або JMS.

СТИЛЬ SOAP

SOAP-повідомлення

Повідомлення SOAP - це просто XML-документ, який може містити чотири елементи:

- Конверт (Envelope)
- Заголовок (Header)
- Тіло (Body)
- Несправність (Fault)

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/">
  <soapenv:Body>
    <getUser>
      <id>1</id>
    </getUser>
  </soapenv:Body>
</soapenv:Envelope>
```

СТИЛЬ SOAP

```
<?xml version="1.0"?>
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
    <soap:Header/>
    <soap:Body>
        <soap:Fault>
            <faultcode>soap:Server</faultcode>
            <faultstring>Query request too large.</faultstring>
        </soap:Fault>
    </soap:Body>
</soap:Envelope>
```

Конверт (Envelope)

- Конверт має бути кореневим елементом XML-документа. У конверті наданий простір імен повідомляє вам, що XML-документ є повідомленням SOAP.

Заголовок (Header)

- Заголовок є необов'язковим елементом, але якщо заголовок присутній, він повинен бути першим дочірнім елементом Envelope. Як і більшість інших заголовків, він містить специфічну для програми інформацію, таку як авторизація, специфічні атрибути SOAP або будь-які атрибути, визначені програмою.

Тіло (Body)

- Тіло містить дані, які потрібно передати одержувачу. Ці дані мають бути у форматі XML та у власному просторі імен.

Несправність (Fault)

- Помилка є необов'язковим елементом, але має бути дочірнім елементом тіла. У повідомленні SOAP може бути лише один елемент помилки. Елемент несправності надає інформацію про помилку та/або стан.

СТИЛЬ GRAPHQL

GraphQL — це гнучкий і потужний запитний архітектурний стиль для API, розроблений компанією Facebook у 2012 році та опублікований у відкритий доступ у 2015 році.

На відміну від традиційного **REST API**, де клієнт отримує заздалегідь визначені дані від сервера, **GraphQL дозволяє клієнту самому визначати, які дані йому потрібні**.

GraphQL використовується для ефективного отримання даних із сервера, дозволяючи **об'єднувати декілька запитів в один** і **запитувати лише потрібні поля**.

GraphQL широко використовується у проєктах, де потрібна **висока продуктивність і гнучкість**.

- ✅ Фронтенд-додатки (React, Vue, Angular)
  ✅ Мобільні застосунки (iOS, Android)
  ✅ Мікросервісна архітектура (інтеграція API)
  ✅ Системи з великою кількістю даних (аналітика, CRM)
  ✅ Соціальні мережі (Facebook, Twitter, Instagram)

СТИЛЬ GRAPHQL

Приклад запиту (Query):

```
{
  user(id: "1") {
    name
    email
    posts {
      title
      content
    }
  }
}
```

Ми отримуємо користувача за id, але тільки ім'я, email та список постів (без зайвих даних).

СТИЛЬ GRAPHQL

У відповідь приходить, наприклад, така інформація:

```
{
  "data": {
    "user": {
      "name": "Олександр",
      "email": "alex@example.com",
      "posts": [
        { "title": "GraphQL vs REST", "content": "GraphQL ефективніший..." }
      ]
    }
  }
}
```

СТИЛЬ GRAPHQL

Наведемо також приклад мутації:

```
mutation {
  createPost(title: "Новий пост", content: "Текст поста") {
    id
    title
  }   }
```

Таким чином створюється новий пост із вказаним заголовком і вмістом. А у відповідь ми отримуємо тільки ID та назву нового поста.

```
{
  "data": {
    "createPost": {
      "id": "101",
      "title": "Новий пост"
    }     }     }
```

СТИЛЬ WEBSOCKET

WebSockets — це **протокол зв’язку у реальному часі**, що забезпечує **двосторонній зв’язок між клієнтом і сервером** через одне постійне з’єднання.

📌 **Як працює?**

1. Клієнт ініціалізує WebSocket-з’єднання (ws:// або wss://).
2. Сервер і клієнт обмінюються даними без необхідності повторного відкриття з’єднання.
3. Дані можуть надходити в обидві сторони **асинхронно** (на відміну від HTTP).

СТИЛЬ WEBSOCKET

Особливості стилю:

✅ **Постійне з’єднання**. Сервер може надсилати оновлення клієнту в реальному часі, без постійних запитів (polling).

✅ **Мінімальний трафік**. Використовує бінарний формат або JSON, передаючи лише корисні дані без зайвих заголовків.

✅ **Підтримка всіма сучасними браузерами**. Дозволяє створювати чати, онлайн-ігри, біржові додатки, моніторинг у реальному часі.

Приклад застосування:

```
import asyncio
import websockets

async def echo(websocket, path):
    async for message in websocket:
        await websocket.send(f"Отримано: {message}")

start_server = websockets.serve(echo, "localhost", 8765)

asyncio.get_event_loop().run_until_complete(start_server)
asyncio.get_event_loop().run_forever()
```

СТИЛЬ REST

REpresentational State Transfer (REST) — це архітектурний стиль, автором якого є американський вчений Рой Томас Філдінг. Він описав REST у розділі 5 своєї докторської дисертації «Архітектурні стилі та проектування мережевих програмних архітектур» у 2000 році. Рой Філдінг визначає REST як гібридний стиль, похідний від кількох мережевих архітектурних стилів, які він описав в інших частинах роботи, «у поєднанні з додатковими обмеженнями, які визначають єдиний інтерфейс з'єднувача».

У роботі Філдінга він встановлює шість обмежень, що застосовуються до елементів в архітектурі:

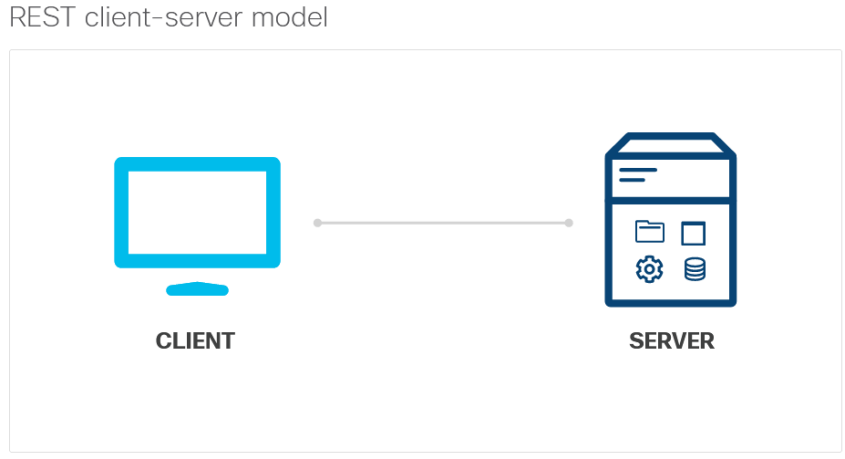
- Client-Server
- Stateless
- Cache
- Uniform Interface
- Layered System
- Code-On-Demand

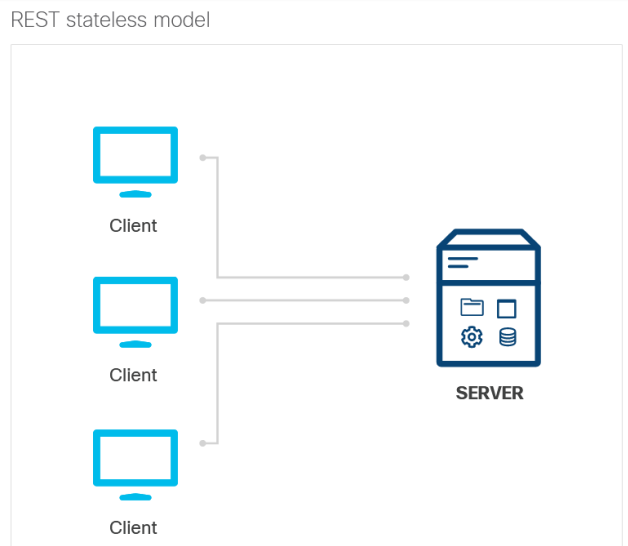
Ці шість обмежень можна застосувати до будь-якого протоколу, і коли вони застосовуються, це означає, що він підтримує REST. Розглянемо їх детальніше.

СТИЛЬ REST

Client-Server

Клієнт і сервер повинні бути незалежними один від одного, що дозволяє створювати клієнта для кількох платформ і спрощувати компоненти на стороні сервера.





Stateless

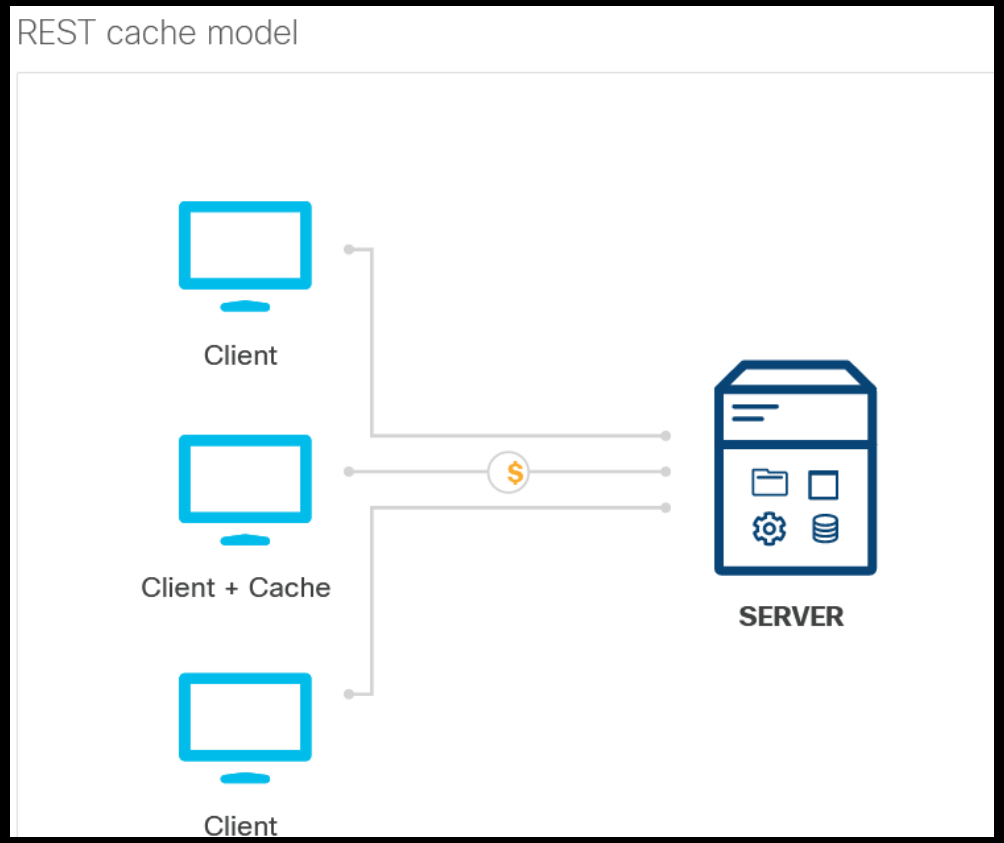
Запити від клієнта до сервера повинні містити всю інформацію, необхідну серверу для виконання запиту. Сервер не може містити стани сеансу.

СТИЛЬ REST

Cache

У відповідях сервера має бути зазначено, кешується відповідь чи не кешується.

Якщо вона кешується, клієнт може використовувати дані з відповіді для подальших запитів.



СТИЛЬ REST

Uniform Interface (Уніфікований інтерфейс)

Інтерфейс між клієнтом і сервером повинен відповідати чотирьом принципам:

1. Ідентифікація ресурсів. Ресурс повинен бути ідентифікований у запиті як окремий об'єкт, до якого сервер буде звертатися та керувати ним. Ресурсом може бути будь-яка інформація, наприклад документ, зображення, особа, набір інших ресурсів тощо. Наприклад, у запиті на зміну пароля для користувача необхідно ідентифікувати окремого користувача.

2. Маніпулювання ресурсами через представлення – клієнт отримує представлення ресурсу від сервера. Це представлення має містити достатньо даних або метаданих, щоб клієнт міг маніпулювати ресурсом. Наприклад, запит, призначений для заповнення профілю користувача, повинен містити інформацію профілю. Представлення може бути точною копією ресурсу на сервері або навіть спрощеною версією ресурсу, але не, наприклад, просто ідентифікатором додаткового ресурсу.

3. Повідомлення з самоописом – кожне повідомлення має містити всю інформацію, яку одержувач може обробити.

Прикладами інформації можуть бути:

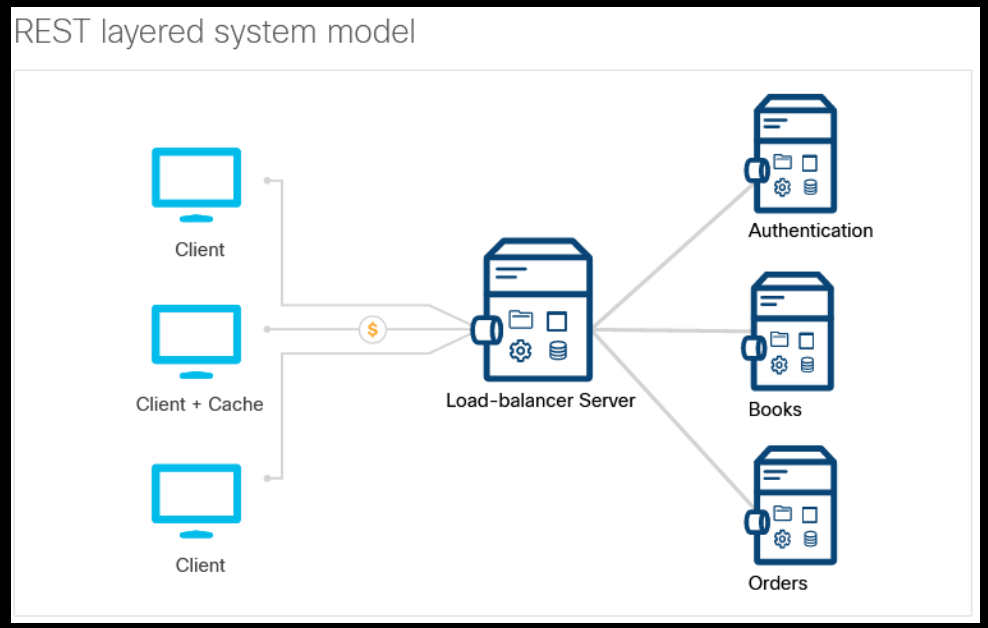
- Тип протоколу
- Формат даних повідомлення
- Запитувана операція

4. Гіпермедіа як механізм стану програми - дані, надіслані сервером, повинні включати додаткові дії та ресурси, доступні клієнту для доступу до додаткової інформації про ресурс.

СТИЛЬ REST

Layered System (Багатошарова система)

Система складається з різних ієрархічних рівнів, в яких кожен рівень надає послуги тільки тому шару, що знаходиться вище. У результаті він споживає послуги з нижнього рівня.



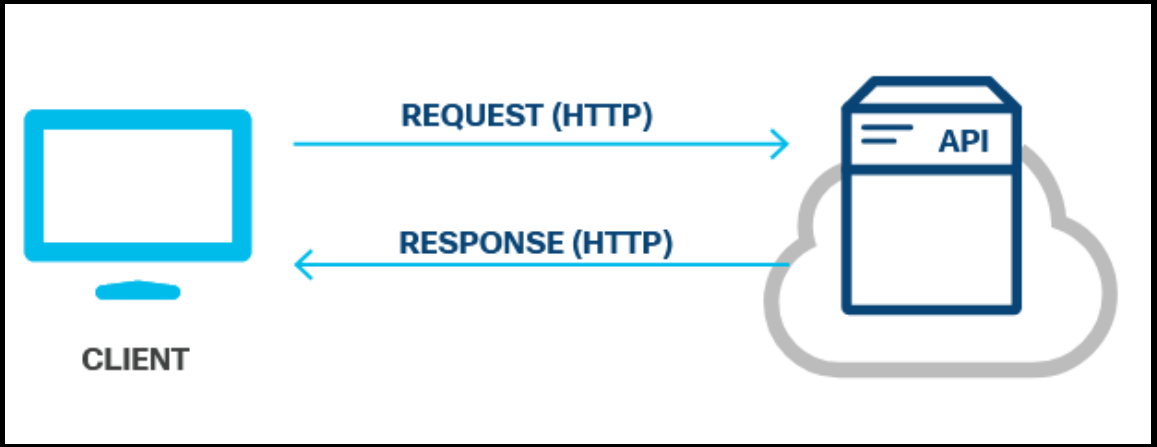
Code-On-Demand (Код за запитом)

Це обмеження є необов'язковим і посилається на той факт, що інформація, яку повертає служба REST, може включати виконуваний код (наприклад, JavaScript) або посилання на такий код, призначений для корисного розширення функціональних можливостей клієнта.

ВСТУП ДО REST APIS

REST API request/response model

API веб-сервісу REST (REST API) — це інтерфейс програмування, який спілкується через HTTP, дотримуючись принципів архітектурного стилю REST.



Оскільки REST API спілкуються через HTTP, вони використовують ті ж концепції, що й протокол HTTP:

- HTTP requests/responses
- HTTP verbs
- HTTP status codes
- HTTP headers/body

REST API REQUESTS

Запити REST API по суті є HTTP-запитами, які відповідають принципам REST. Ці запити є способом для програми (клієнта) попросити сервер виконати функцію. Оскільки це API, ці функції попередньо визначені сервером і повинні відповідати наданій специфікації.

REST API запити складаються з чотирьох компонент:

- Uniform Resource Identifier (URI)
- HTTP Method
- Header
- Body

Uniform Resource Identifier (URI)

Uniform Resource Identifier (URI) іноді плутають з Uniform Resource Locator (URL). URI визначає з яким ресурсом хоче взаємодіяти клієнт. Запит REST повинен ідентифікувати запитуваний ресурс; ідентифікація ресурсів для REST API зазвичай є частиною URI .

URI по суті є таким же форматом, що й URL-адреса, яку ви використовуєте у браузері для переходу на веб-сторінку.

Синтаксис URI складається з таких компонентів у цьому конкретному порядку:

- Scheme
- Authority
- Path
- Query

REST API REQUESTS

Коли ви з'єднаєте компоненти разом, URI буде виглядати так: `scheme:[//authority][/path][?query]`

Scheme

Схема визначає, який протокол HTTP слід використовувати. Для REST API є два варіанти:

1)http – відкрите з'єднання 2) https – захищене з'єднання

Authority

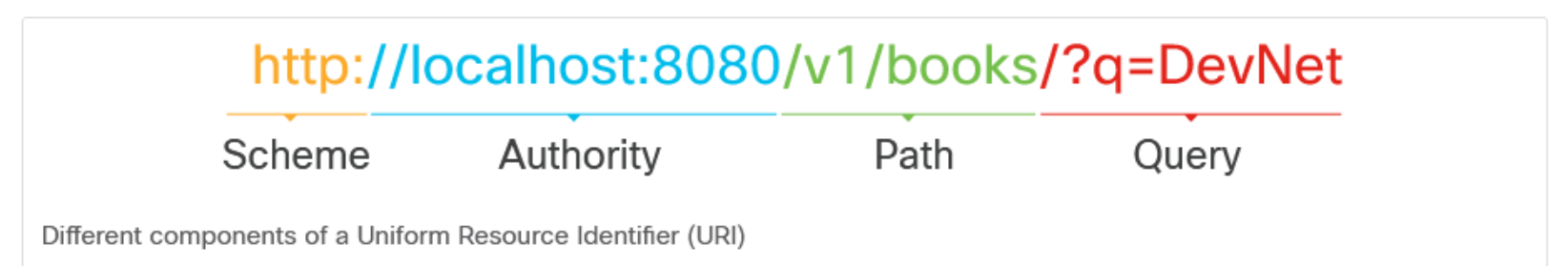
Authority (повноваження), або призначення, складається з двох частин, перед якими стоять дві косі риски (//)

- Host
- Port

Хост — це ім'я хоста або IP-адреса сервера, який надає REST API (веб-сервіс). Порт - це кінцева точка зв'язку або номер порту, який пов'язаний з хостом. Перед портом завжди стоїть двокрапка (:) *Зауважте, що якщо сервер використовує порт за замовчуванням — 80 для HTTP і 443 для HTTPS — порт може бути не вказано в URI.*

Path

Для REST API шлях зазвичай відомий як шлях до ресурсу і представляє місце розташування ресурсу, даних або об'єкта, якими потрібно маніпулювати на сервері. Шляху передує косою рискою (/) і може складатися з кількох сегментів, розділених косою рискою (/)



REST API REQUESTS

Query

Запит (query), який містить параметри запиту, є необов'язковим. Запит надає додаткові відомості щодо обсягу, фільтрації або роз'яснення запиту. Якщо запит присутній, перед ним ставиться знак питання (**?**). Немає спеціального синтаксису для параметрів запиту, але він зазвичай визначається як набір пар ключ-значення, розділених амперсандом (**&**). Наприклад:

```
http://example.com/update/person?id=42&email=person%40example.com
```

HTTP method

REST API використовують стандартні методи HTTP, також відомі як HTTP дієслова (**HTTP verbs**), як спосіб повідомити веб-сервісу, яка дія запитується для даного ресурсу. Немає стандарту, який визначає, який метод HTTP зіставляється з яким дієсловом, але запропоноване зіставлення виглядає так:

HTTP Method	Action	Description
POST	Create	Створити новий об'єкт або ресурс.
GET	Read	Отримати інформацію про ресурс із системи.
PUT	Update	Заміна або оновлення існуючого ресурсу.
PATCH	Partial Update	Оновлення деяких деталей з наявного ресурсу .
DELETE	Delete	Видалення ресурсу із системи.

REST API REQUESTS

Header

API REST використовують стандартний формат заголовка HTTP для передачі додаткової інформації між клієнтом і сервером, але ця додаткова інформація необов'язкова. Заголовки HTTP відформатовані як пари ім'я-значення, розділені двокрапкою (:) , **[ім'я]:[значення]**. Визначено деякі стандартні заголовки HTTP, але веб-сервіс, який приймає запит REST API, може визначити спеціальні заголовки для прийняття.

REST API REQUESTS

Існує два типи заголовків: заголовки запитів і заголовки сутностей.

Заголовки запитів (Request headers)

Заголовки запиту містять додаткову інформацію, яка не стосується змісту повідомлення. Наприклад, ось типовий заголовок запиту, який ви можете знайти для запиту REST API:

Ключ	Приклад значення	Опис
Authorization	Basic dmFncmFudDp2YWdyYW50	Надає облікові дані для авторизації запиту

Заголовки сутностей (Entity headers)

Заголовки сутностей — це додаткова інформація, яка описує зміст тіла повідомлення. Ось типовий заголовок сутності, який ви можете знайти для запиту REST API:

Ключ	Приклад значення	Опис
Content-Type	application/json	Уточнює формат даних у тілі (body)

REST API REQUESTS

Тіло (Body)

Тіло запиту REST API містить дані, що стосуються ресурсу, яким клієнт хоче маніпулювати. Запити REST API, які використовують метод HTTP POST, PUT і PATCH, зазвичай містять тіло. Залежно від методу HTTP, тіло є необов'язковим, але якщо дані надаються в тілі, тип даних необхідно вказати в заголовку за допомогою ключа Content-Type. Деякі API створені для прийому кількох типів даних у запиті.

REST API Responses

Відповіді REST API (REST API Responses) по суті є відповідями HTTP. Ці відповіді передають результати HTTP-запиту клієнта. Відповідь може містити дані, означати, що сервер отримав його запит, або навіть повідомляти клієнту про проблему з його запитом.

Відповіді REST API подібні до запитів, але складаються з трьох основних компонентів:

- HTTP Status
- Header
- Body

HTTP status

API REST використовують стандартні коди статусу HTTP у відповіді, щоб повідомити клієнта, чи був запит успішним чи невдалим. Сам код статусу HTTP може допомогти клієнту визначити причину помилки, а іноді може надати пропозиції щодо вирішення проблеми. Коди статусу HTTP завжди тризначні. Перша цифра — це категорія відповіді. Дві інші цифри не мають значення, але зазвичай призначаються по порядку.

Існує п'ять різних категорій:

1xx - Informational

2xx - Success

3xx - Redirection

4xx - Client Error

5xx - Server Error

REST API REQUESTS

1xx - informational

Відповіді з кодом 1xx призначені для інформаційних цілей, що вказують на те, що сервер отримав запит, але не закінчив його обробку. Клієнт повинен очікувати повної відповіді пізніше. Ці відповіді зазвичай не містять тіла.

2xx - success

Відповіді з кодом 2xx означають, що сервер отримав та прийняв запит. Для синхронних API ці відповіді містять запитані дані в тілі (якщо є). Для асинхронних API відповіді зазвичай не містять тіла, а код статусу 2xx є підтвердженням того, що запит отримано, але його все ще потрібно виконати.

3xx - redirection

Відповіді з кодом 3xx означають, що клієнт повинен виконати додаткову дію, щоб запит було виконано. У більшості випадків потрібно використовувати іншу URL-адресу. Залежно від того, як було викликано REST API, користувач може бути автоматично перенаправлений без будь-яких дій вручну.

4xx - client error

Відповіді з кодом 4xx означають, що запит містить помилку, таку як неправильний синтаксис або неправильне введення, що перешкоджає виконанню запиту. Клієнт повинен вжити заходів для вирішення цих проблем, перш ніж повторно надіслати запит.

5xx - server error

Відповіді з кодом 5xx означають, що сервер не може виконати запит, навіть якщо сам запит є дійсним. Залежно від того, який саме код статусу 5xx, клієнт може повторити запит пізніше.

REST API REQUESTS

Поширені коди статусу HTTP

HTTP Status Code	Status Message	Описання
200	OK	Запит був успішним і зазвичай містить корисне навантаження (тіло)
201	Created	Запит виконано, і запитуваний ресурс створено
202	Accepted	Запит прийнято до обробки та обробляється
400	Bad Request	Запит не буде оброблено через помилку із запитом
401	Unauthorized	Запит не має дійсних облікових даних автентифікації для виконання запиту
403	Forbidden	Запит зрозумлий, але сервер відхилив його
404	Not Found	Запит не може бути виконаний, оскільки на сервері не знайдено шлях до ресурсу запиту
500	Internal Server Error	Запит не може бути виконаний через помилку сервера
503	Service Unavailable	Запит не може бути виконаний, оскільки наразі сервер не може обробити запит

REST API REQUESTS

Заголовок

Так само, як і запит, заголовок відповіді також використовує стандартний формат заголовка HTTP і також є необов'язковим. Заголовок у відповіді має надати додаткову інформацію між сервером і клієнтом у форматі пари ім’я-значення, що розділяється двокрапкою (:), **[ім’я]:[значення]**.

Існує два типи заголовків: заголовки відповіді та заголовки об’єктів.

Заголовки відповідей

Заголовки відповіді містять додаткову інформацію, яка не стосується змісту повідомлення.

Деякі типові заголовки відповіді, які ви можете знайти для запиту REST API, включають:

Ключ	Приклад	Опис
Set-Cookie	JSESSIONID=30A9DN810FQ428P; Path=/	Використовується для надсилання файлів cookie з сервера
Cache-Control	Cache-Control: max-age=3600, public	Вказує директиви, яким ПОВИННІ підкорятися всі механізми кешування

REST API REQUESTS

Заголовки сутностей

Заголовки сутностей — це додаткова інформація, яка описує зміст тіла повідомлення.

Один загальний заголовок сутності визначає тип даних, які повертаються:

Ключ	Приклад	Опис
Content-Type	application/json	Вказує формат даних у тілі

Body

Тілом відповіді REST API є дані, які клієнт запитав у запиті REST API. Тіло є необов'язковим, але якщо дані вказано в тілі, тип даних вказується в заголовку за допомогою ключа Content-type. Якщо запит REST API був невдалим, сервер може надати додаткову інформацію про проблему або дії, які необхідно виконати, щоб запит був успішним.

Пагінація відповідей

Деякі API, наприклад API пошуку, можуть вимагати надсилання великої кількості даних у відповіді. Для зменшення використання пропускної здатності в мережі, ці API розбиватимуть дані відповіді на сторінки. Пагінація відповіді дозволяє розбити дані на частини. Більшість API, які реалізують розбивку на сторінки, дозволять запитувачу вказати, скільки елементів він хоче у відповіді.
Оскільки є кілька фрагментів, API також повинен дозволити запитувачу вказати, який фрагмент він хоче. Для API не існує стандартного способу реалізації розбиття на сторінки, але більшість реалізацій використовують параметр запиту, щоб вказати, яку сторінку повертати у відповіді.

REST API REQUESTS

Стислі дані відповіді

Коли серверу потрібно надіслати дуже великі обсяги даних, які неможливо розбити на сторінки, стислі дані є ще одним способом зменшити навантаження на пропускну здатність. Це стиснення даних може бути запитано клієнтом через сам запит API. Щоб запитати стиснення даних, запит має додати поле ***Accept-Encoding*** до заголовка запиту.

Прийняті значення:

- gzip
- compress
- deflate
- br
- identity
- *

Якщо сервер не може надати жоден із запитуваних типів стиснення, він надішле відповідь із кодом статусу 406 – Неприйнятно

Якщо сервер виконує стиснення, він надішле відповідь зі стисненими даними і додасть поле ***Content-Encoding*** до заголовка відповіді. Значення Content-Encoding — це тип стиснення, який використовувався, що дозволяє клієнту належним чином розпакувати дані.

REST API REQUESTS

Використання діаграм послідовності з REST API

Діаграми послідовності використовуються для пояснення послідовності обмінів або подій. Вони забезпечують сценарій упорядкованого набору подій. Їх також називають діаграмами подій. Хоча один запит REST API може служити для отримання інформації або для ініціювання змін у системі, частіше взаємодія з певною службою REST API буде послідовністю запитів. Тому діаграми послідовності часто використовуються для пояснення запитів/відповідей в REST API та асинхронної діяльності.

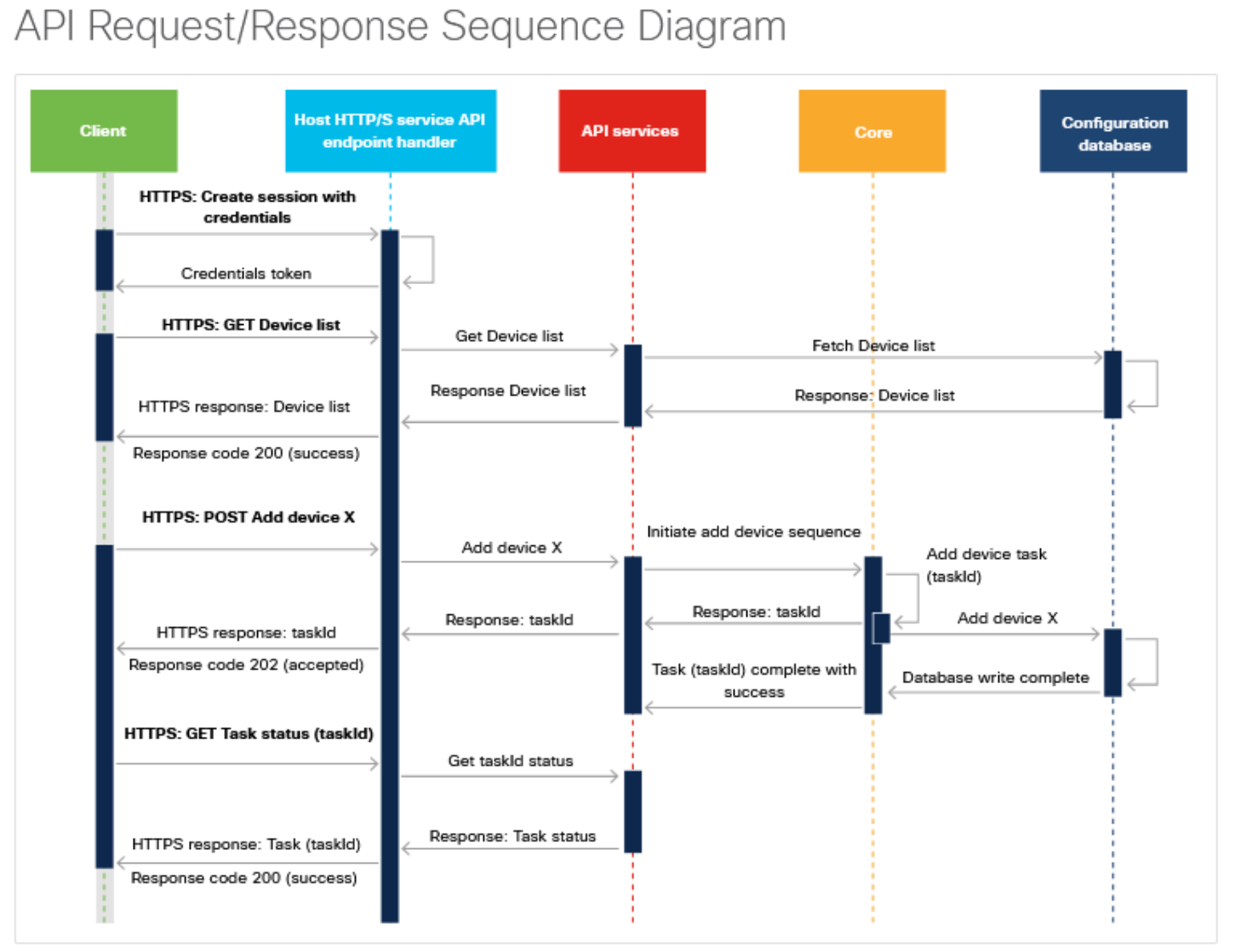
Формалізовані діаграми послідовності тісно пов'язані та вважаються підмножиною стандартизованої системи моделювання, відомої як Unified Modeling Language (UML). UML включає стандартизовані підходи до інших аспектів статичних ресурсів і процесів, включаючи стандартизовані способи побудови діаграм і пояснення користувацьких інтерфейсів, визначення класів і об'єктів, а також поведінку взаємодії. Діаграми послідовності є одним із способів діаграми поведінки взаємодії.

У стандартній діаграмі послідовності, вісь Y є впорядкованим і немасштабованим часом, з нульовим часом угорі і часом, що зростає до низу.

Вісь X складається з ліній подій, представлених вертикальними лініями із заголовком, і обмінів або повідомлень, представлених горизонтальними стрілками. Лінія події — це будь-який елемент, який може взаємодіяти, одержуючи або генеруючи повідомлення. За умовою, інтерфейсні ініціатори послідовності, такі як користувач, клієнт або веб-браузер, розміщуються в лівій частині діаграми. Такі елементи, як файлові системи, бази даних, постійне сховище тощо, розміщуються праворуч. Проміжні служби, такі як веб-сервер або кінцеві точки API, розташовані посередині.

Одним із корисних аспектів діаграм послідовності є те, що користувачі діаграм можуть зосередитися на взаємодії лише двох елементів лінії події. Наприклад, хоча інша частина діаграми може допомогти встановити контекст, користувачі REST API можуть зосередитися на взаємодії між клієнтом і інтерфейсом або, як показано нижче, обробником кінцевої точки API служби HTTP/S хосту.

REST API REQUESTS



Клієнтом **Client** зліва може бути програма зі сценарієм Python, тестове середовище POSTMAN або будь-який інший генератор запитів API. Клієнт надішле запит у форматі HTTPS до інтерфейсу сервера під назвою **Host HTTP/S Service API** кінцевої точки обробки. Цей компонент зазвичай передає запит відповідному обробнику API, який називається **API Services**. Праворуч від API Services розташований елемент із простою назвою **Core**, який є універсальним елементом для логіки або платформи первинної обробки.

Служби API можуть обробляти деякі запити безпосередньо або можуть інтерпретувати, а потім пересилати деякі або всі запити в **Core**. Крайній правий стовпець, показаний тут як **Configuration Database**, може бути будь-яким постійним сховищем або навіть процесом обміну повідомленнями чи чергою для зв'язку з іншою системою.

У цьому прикладі показано три окремі послідовності: створити сеанс, отримати пристрої та створити пристрій.

REST API REQUESTS

Розглянемо послідовності з прикладу по-черзі.

Create Session: Початковий запит позначено як **HTTPS: Create Session w/credentials**. Тут логіка створення сеансу HTTPS знаходиться у інтерфейсі обробника кінцевої точки API Host HTTP/S Service. (Це показано графічно за допомогою стрілки, яка повертається назад.)

Get Devices: Другий запит від клієнта — запит на список пристроїв з платформи. У цій послідовності запит пересилається до модуля API Services, який потім містить логіку для безпосереднього запиту бази даних конфігурації, отримання списку пристроїв і повернення списку обробнику кінцевої точки. Потім обробник обертає вміст у відповідь HTTPS і повертає його клієнту разом з кодом статусу HTTP, що вказує на успіх .

Ці перші дві послідовності демонструють синхронний обмін, у якому після запиту слідує відповідь, а завдання повністю завершується з успіхом або невдачею.

REST API REQUESTS

Create Device: Третя послідовність починається із запиту POST на створення пристрою. У цьому випадку запит переміщується до служб API, які потім пересилають запит до основної логіки, яка потім починає працювати над запитом. Служби API чекають лише повідомлення від ядра, що підтверджує, що процес створення пристрою почався. Core надає дескриптор (TaskId), який ідентифікує роботу та дозволяє відстежувати, чи була робота завершена. Значення TaskId поширюється у відповідь клієнту. Відповідь HTTP повідомляє клієнту лише дескриптор (TaskId) і що запит було ініційовано з кодом відповіді 202 (Прийнято). Це свідчить про те, що запит прийнято, і що робота зараз триває.

Основна логіка продовжує виконуватися. Він оновлює базу даних конфігурації, а потім повідомляє служби API, коли це буде завершено. Через деякий час клієнт може підтвердити, що завдання виконано. Клієнт робить це за допомогою запиту Статус завдання. Служби API потім можуть відповісти інформацією про статус завершення та статус успіху чи невдачі для цього завдання. Оскільки фактична запитувана робота не була виконана до отримання відповіді клієнту, ця взаємодія вважається асинхронною.

АУТЕНТИФІКАЦІЯ REST API

З міркувань безпеки більшість API REST вимагає аутентифікації, щоб випадкові користувачі не могли створювати, оновлювати або видаляти інформацію неправильно або зловмисно, а також отримати доступ до інформації, яка не повинна бути загальнодоступною.

Без автентифікації REST API дозволяє будь-кому отримати доступ до функцій і системних служб, які були відкриті через інтерфейс. Вимагання автентифікації — це те саме поняття, що й вимога введення імені користувача/паролю для доступу до сторінки адміністрування програми. Деякі API не вимагають аутентифікації.

Ці API зазвичай доступні лише для читання і не містять критичної чи конфіденційної інформації.

АУТЕНТИФІКАЦІЯ REST API

Authentication vs Authorization

Важливо розуміти різницю між аутентифікацією та авторизацією під час роботи з REST API, оскільки ці два терміни часто використовуються як взаємозамінні або неправильно. Знання різниці допоможе усунути будь-які проблеми, пов'язані з безпекою запиту REST API.

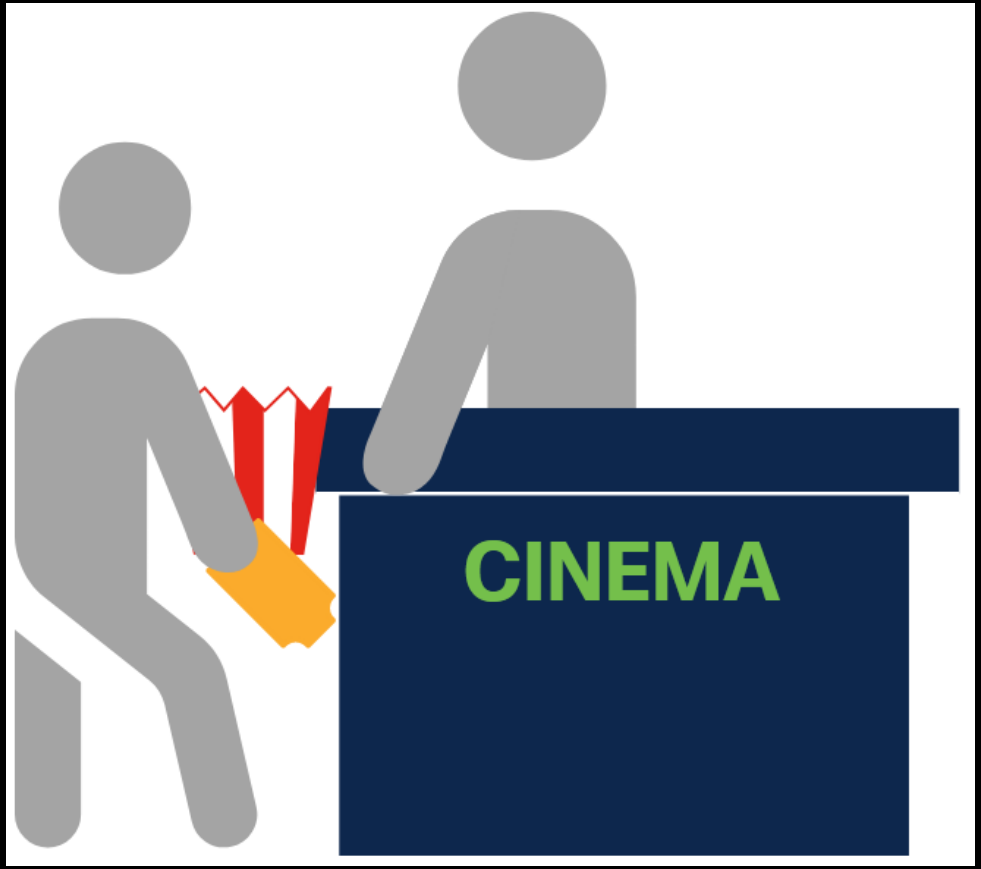
Authentication

Аутентифікація - це акт перевірки особистості користувача. Користувач доводить, що він є тим, за кого себе видає. *Наприклад, коли ви йдете в аеропорт, ви повинні пред'явити своє державне посвідчення особи або використовувати біометричні дані, щоб підтвердити, що ви є особою, за яку ви себе видаєте.*

АВТОРИЗАЦІЯ REST API

Authorization

Авторизація — це користувач, який підтверджує, що він має права виконувати запитувану дію на цьому ресурсі. *Наприклад, коли ви йдете на концерт, вам потрібно лише показати квиток, щоб підтвердити, що вас пускають. Вам не обов'язково підтверджувати свою особу.*



МЕХАНІЗМИ АУТЕНТИФІКАЦІЇ REST API

Поширені типи механізмів аутентифікації включають базовий, носій і ключ API. Основна аутентифікація

Базова автентифікація використовує стандартну базову схему аутентифікації HTTP. Basic Auth передає облікові дані як пари ім'я користувача/пароль, розділені двокрапкою (:) і закодовані за допомогою Base64.

У запиті REST API основна інформація про автентифікацію буде надана в заголовку:

```
Authorization: Basic <username>:<password>
```

Basic Auth є найпростішим механізмом аутентифікації. Це надзвичайно небезпечно, якщо він не поєднується із запитами, які використовують HTTPS, а не HTTP. Хоча облікові дані закодовані, вони не зашифровані. Розшифрувати облікові дані та отримати пару ім'я користувача/пароль легко.

Аутентифікація носія (Bearer authentication)

Аутентифікація носія, також відома як аутентифікація за маркером, використовує стандартну схему автентифікації переносника HTTP. Він є більш безпечним, ніж базова автентифікація, і зазвичай використовується з OAuth і єдиним входом (Single Sign-On - SSO). Аутентифікація носія використовує маркер носія, який є рядком, згенерованим сервером аутентифікації, таким як служба ідентифікації (IdS).

У запиті REST API інформація Bearer Auth буде надана в заголовку:

```
Authorization: Bearer <bearer token>
```

Так само, як і базова автентифікація, аутентифікація на носії повинна використовуватися з HTTPS.

API KEY

Ключ API, також званий маркером API, — це *унікальний буквено-цифровий рядок, який генерується сервером і призначається користувачеві*. Щоб отримати унікальний ключ API, користувач зазвичай входить на портал, використовуючи свої облікові дані. Цей ключ зазвичай призначається один раз і не буде генеруватися повторно. Усі запити REST API для цього користувача мають надавати призначений ключ API як форму аутентифікації. Так само, як і інші типи аутентифікації, ключі API безпечні лише тоді, коли використовуються з HTTPS. Ключі API призначені як механізм аутентифікації, але зазвичай використовуються неправильно як механізм авторизації.

Існує два типи ключів API: відкритим та приватні.

Відкритий (public) ключ API може бути спільний, що дає цьому користувачеві доступ до підмножини даних і API. Не надавайте стороннім **приватний ключ**, оскільки він може бути засобом ідентифікації. Термін дії більшості ключів API не закінчується, і якщо ключ не може бути відкликаний або повторно створений, якщо він розповсюджений або зламаний, будь-хто, хто має цей ключ, може отримати необмежений доступ до системи, як і ви.

Запит REST API може надати ключ API кількома різними способами:

Рядок запиту (Query string): рекомендовано лише для відкритих ключів API

- Заголовок: використовує ключ авторизації або спеціальний *Authorization: <API Key> або Authorization: APIkey <API Key> або APIkey: <API Key>*
- Основні дані: використовує унікальний ключ як ідентифікатор *Content-Type: application/json*
- Cookie: використовує унікальний ключ як ідентифікатор *Cookie: API_KEY=<API Key>*

МЕХАНІЗМИ АВТОРИЗАЦІЇ

Відкрита авторизація, також відома як **OAuth**, поєднує автентифікацію з авторизацією. OAuth був розроблений як рішення для незахищених механізмів аутентифікації. З підвищеною безпекою порівняно з іншими варіантами, зазвичай це рекомендована форма аутентифікації/авторизації для API REST.

Існує дві версії OAuth, які називаються просто **OAuth 1.0** і **OAuth 2.0**. Більшість сучасних API REST реалізують OAuth 2.0. *OAuth 2.0 не має зворотної сумісності*.

В платформі авторизації вмзначається OAuth 2.0, що «фреймворк авторизації OAuth 2.0 дає змогу програмі сторонніх розробників отримувати обмежений доступ до служби HTTP від імені власника ресурсу шляхом організації взаємодії власника ресурсу та служби HTTP, або дозволивши програмі третьої сторони отримати доступ від свого імені».

По суті, OAuth 2.0 дає змогу попередньо зареєстрованим програмам отримувати авторизацію для виконання запитів REST API від імені користувача без необхідності передавати свої облікові дані самій програмі. OAuth дозволяє користувачеві надавати облікові дані безпосередньо серверу авторизації, як правило, постачальнику ідентифікації (IdP) або службі ідентифікації (IdS), щоб отримати маркер доступу, який можна спільно використовувати з програмою.

Цей процес отримання токена називається потоком. Потім програма використовує цей маркер в REST API як аутентифікацію носія. Потім веб-сервіс для REST API перевіряє сервер авторизації, щоб переконатися, що маркер дійсний, а запитувач уповноважений виконувати запит.

ОБМЕЖЕННЯ ШВИДКОСТІ **(RATE LIMITS)**

API REST дають змогу будувати складні взаємодії. Використання обмеження швидкості API — це спосіб веб-сервісу контролювати кількість запитів, які користувач або програма може зробити за певну одиницю часу. Запровадження обмежень швидкості є найкращою практикою для загальнодоступних і необмежених API.

Обмеження швидкості допомагає:

- уникнути перевантаження сервера від занадто великої кількості запитів одночасно
- забезпечити краще обслуговування та час відповіді всім користувачам
- захист від атаки відмови в обслуговуванні (DoS).

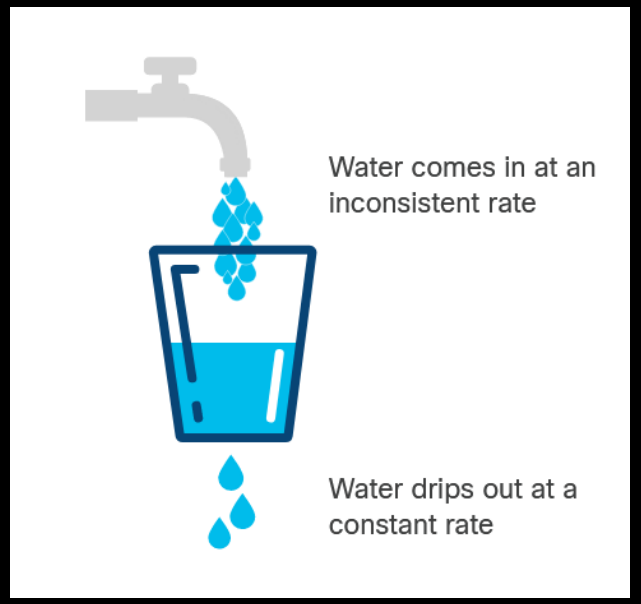
Користувачі API повинні розуміти різні реалізації алгоритму обмеження швидкості, щоб уникнути перевищення меж, але програми також повинні аккуратно обробляти ситуації, коли ці обмеження перевищуються.

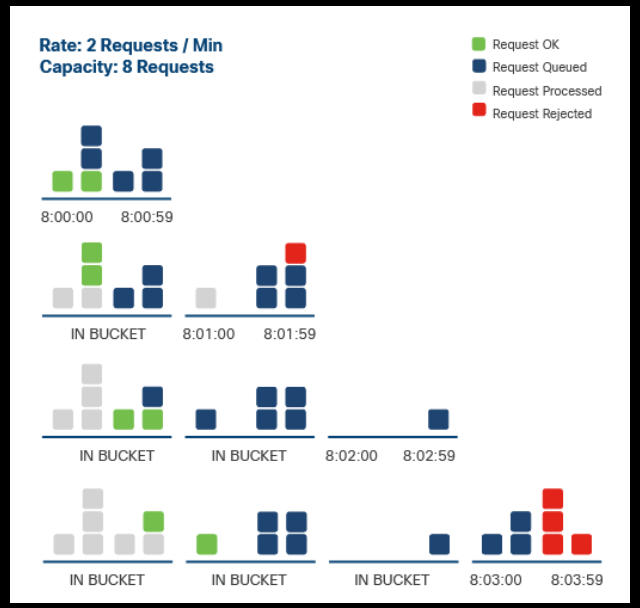
АЛГОРИТМИ ОБМЕЖЕННЯ ШВИДКОСТІ

Стандартного способу застосування обмеження швидкості не існує, але загальні алгоритми включають:

- «Діряве відро» (Leaky bucket)
- «Відро токенів» (Token bucket)
- Фіксований лічильник вікна (Fixed window counter)
- Ковзаючий лічильник вікна (Sliding window counter)

«Діряве відро»

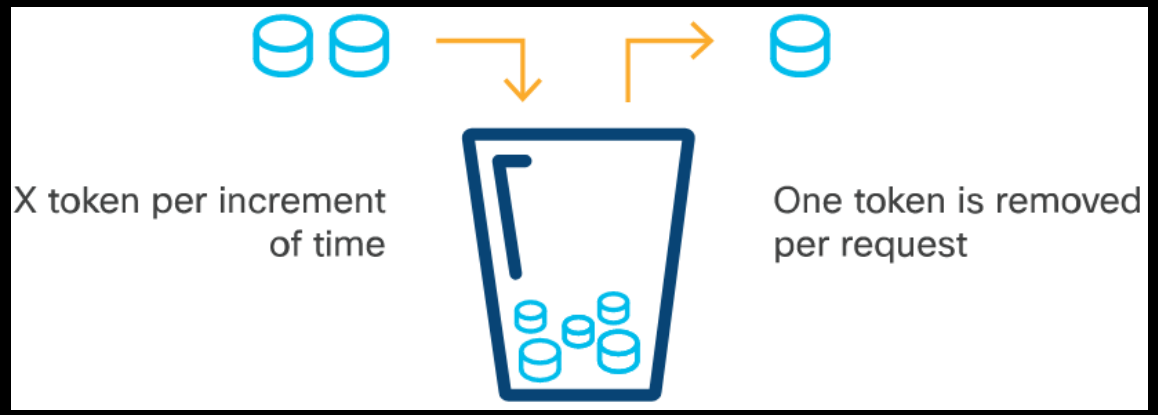


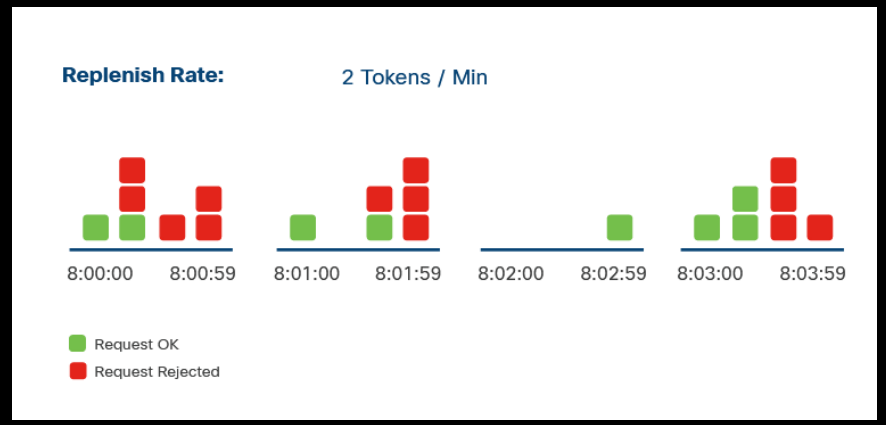


Алгоритм «діряве відро» поміщає всі вхідні запити в чергу запитів у тому порядку, в якому вони були отримані. Вхідні запити можуть надходити будь-якою швидкістю, але сервер оброблятиме запити з черги з фіксованою швидкістю. Якщо черга запитів заповнена, запит відхиляється. За допомогою цього алгоритму клієнт повинен бути готовий до затримки відповідей або відхилення запитів.

АЛГОРИТМИ ОБМЕЖЕННЯ ШВИДКОСТІ

«Відро токенів»



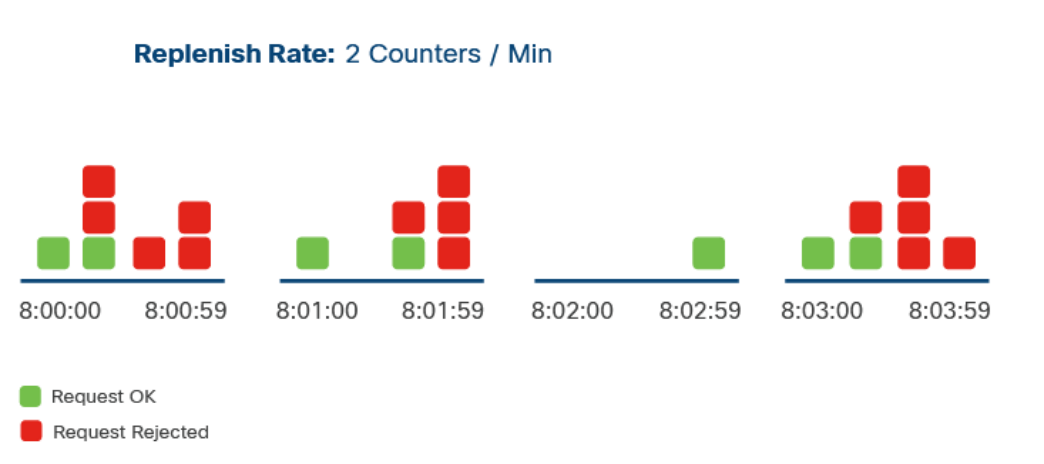


Алгоритм «Відра токенів» дає кожному користувачеві певну кількість токенів, які вони можуть використовувати протягом певного проміжку часу, і ці токени накопичуються, доки не будуть використані. Коли клієнт робить запит, сервер перевіряє сегмент, щоб переконатися, що він містить принаймні один токен. Якщо так, він видаляє цей токен і обробляє запит. Якщо токен недоступний, він відхиляє запит. Усі запити, зроблені до поповнення токена, будуть відхилені. Після поповнення токенів користувач може знову робити запити. Наприклад, API, що використовує алгоритм Token Bucket, встановлює обмеження швидкості в 10 запитів на клієнта на годину. Якщо клієнт зробить 11 запитів протягом години, 11 запит буде відхилено, оскільки не залишилося токенів. З іншого боку, якщо клієнт не робить жодних запитів протягом 6 годин, він може зробити 60 запитів одночасно, оскільки ці токени накопичилися. За допомогою цього алгоритму клієнт повинен обчислити, скільки токенів він має на даний момент, щоб уникнути відхилених запитів. Він також повинен обробляти потенційні відхилені запити, вбудовуючи механізм повторних спроб, коли токени поповнюються.

АЛГОРИТМИ ОБМЕЖЕННЯ ШВИДКОСТІ

Алгоритм **лічильника фіксованого вікна** подібний до сегмента маркерів, за винятком двох основних відмінностей:

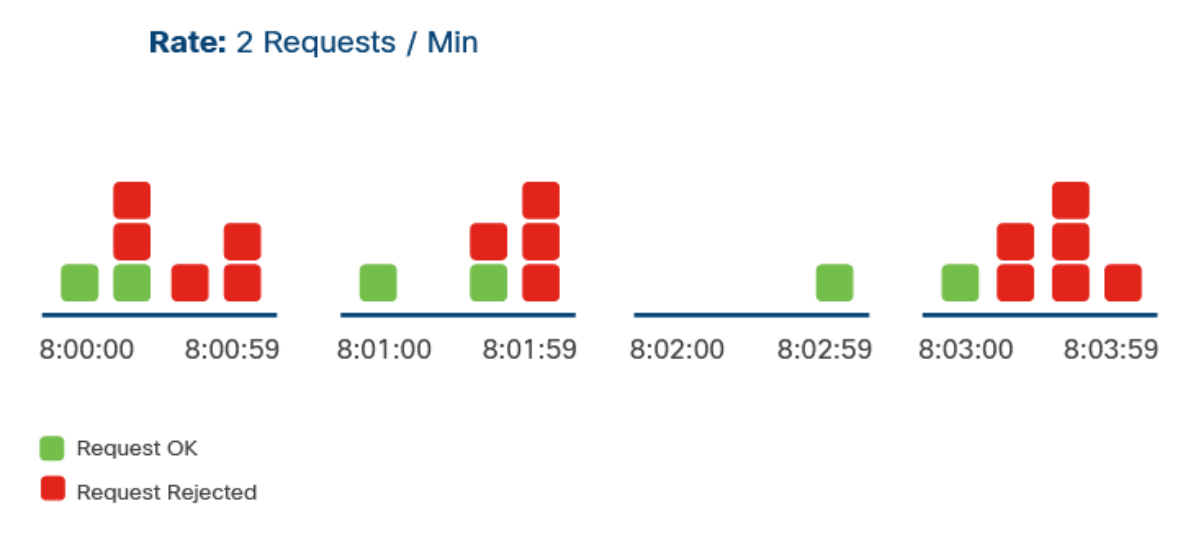
- Він використовує лічильник, а не колекцію маркерів.
- Лічильник не може бути накопичений.



Для цього алгоритму фіксованому вікну часу призначається лічильник, який відображає, скільки запитів можна обробити за цей період. Коли сервер отримує запит, лічильник поточного періоду часу перевіряється, щоб переконатися, що він не дорівнює нулю. Коли запит оброблено, лічильник знімається. Якщо обмеження для цього періоду часу буде дотримано, усі наступні запити протягом цього періоду часу будуть відхилені. Коли почнеться наступне вікно часу, лічильник буде повернуто до попередньо визначеного лічильника, і запити можна буде обробляти знову. Щоб повернутися до нашого попереднього прикладу з 10 запитами на годину з використанням цього алгоритму, 11-й запит за годину все одно буде відхилено, але після 6 годин без жодних запитів клієнт все ще може зробити лише 10 запитів за одну годину, тому що вони " невикористані» запити не накопичувалися. За допомогою цього алгоритму клієнт повинен знати, коли вікно часу починається і закінчується, щоб він знав, скільки запитів можна зробити за цей проміжок часу. Подібно до алгоритму «відра токенів», клієнт повинен побудувати механізм повторних спроб, щоб він міг повторити запити, коли почнеться наступне вікно часу.

АЛГОРИТМИ ОБМЕЖЕННЯ ШВИДКОСТІ

Алгоритм **лічильника ковзаючого вікна** дозволяє виконувати фіксовану кількість запитів за встановлену тривалість часу. Цей період часу не є фіксованим вікном, і лічильник не поповнюється, коли вікно починається знову. У цьому алгоритмі сервер зберігає мітку часу, коли зроблено запит.



Коли зроблено новий запит, сервер підраховує, скільки запитів вже було зроблено від початку вікна до поточного часу, щоб визначити, чи слід обробити запит чи відхилити. Наприклад, якщо швидкість становить п'ять запитів на хвилину, коли сервер отримує новий запит, він перевіряє, скільки запитів було зроблено за останні 60 секунд. Якщо вже подано п'ять запитів, то новий запит буде відхилено. За допомогою цього алгоритму клієнту не потрібно знати, коли починається і закінчується вікно часу. Потрібно лише переконатися, що ліміт ставки не було перевищено на момент запиту. Клієнт також повинен розробити спосіб затримки запитів, якщо це необхідно, щоб вона не перевищувала дозволену швидкість, і, звичайно, врахувати відхилені запити.

ОБМЕЖЕННЯ ШВИДКОСТІ (RATE LIMITS)

Як дізнатись який ліміт швидкості використовується?

Документація API зазвичай містить деталі обмеження швидкості та одиниці часу. Крім того, багато API додають деталі про обмеження швидкості в заголовок відповіді. Оскільки стандарту немає, пара ключ-значення, що використовується в заголовку, може відрізнятися між API.

Деякі часто використовувані ключі включають:

X-RateLimit-Limit: максимальна кількість запитів, які можна зробити за певну одиницю часу

X-RateLimit-Remaining: кількість запитів, що залишилися, які запитувач може зробити у поточному вікні обмеження швидкості

X-RateLimit-Reset: час скидання вікна обмеження швидкості Клієнт може використовувати цю інформацію, щоб відстежувати, скільки ще запитів API він може зробити у поточному вікні, допомагаючи клієнту уникнути перевищення ліміту швидкості.

Перевищення ліміту швидкості

Коли ліміт швидкості перевищено, сервер автоматично відхиляє запит і надсилає відповідь HTTP, інформуючи користувача. Крім того, зазвичай відповідь, що містить помилку «перевищено межу швидкості», також містить змістовний код статусу HTTP.

На жаль, оскільки для цієї взаємодії не існує стандарту, сервер може вибрати, який код статусу надіслати. Найбільш часто використовувані коди статусу HTTP: 429: Забагато запитів або 403: Заборонено.

WEBHOOK

Webhook — це зворотний виклик HTTP або HTTP POST до вказаної URL-адреси, яка сповіщає вашу програму про те, що в одному з ваших ресурсів на платформі відбулася певна активність або «подія». Концепція проста. Подумайте про те, щоб запитати когось «негайно скажи мені, якщо X станеться». Цей «хтось» є постачальником вебхуку, а ви – програмою. Веб-хуки дозволяють програмам отримувати дані в режимі реального часу, оскільки вони ініціюються певними діями або подіями.

За допомогою вебхуків програми є більш ефективними, оскільки їм більше не потрібно мати механізм опитування.

Механізм опитування – це спосіб багаторазового запиту інформації від служби, доки не буде виконана умова. Уявіть, що вам потрібно запитати когось знову і знову: «Чи вже відбувся X? Дратує, правда? Це опитування. Опитування погіршує продуктивність як клієнта, так і сервера через багаторазову обробку запитів і відповідей. Крім того, опитування не відбувається в режимі реального часу, оскільки опитування відбуваються через фіксовані проміжки часу. Якщо подія відбувається відразу після останнього опитування, ваша програма не дізнається про змінений статус, доки не закінчиться інтервал опитування та не відбудеться наступне опитування.

Вебхуки також відомі як зворотні API, оскільки програми підписуються на сервер веб-хуку, реєструючись у постачальника вебхуку. Під час цього процесу реєстрації програма надає URI для виклику сервером, коли відбувається цільова активність або подія. Цей URI зазвичай є API на стороні програми, яку сервер викликає, коли запускається вебхук.

Коли спрацьовує вебхук, сервер надсилає сповіщення, стаючи абонентом, і робить запит до наданого URI. Цей URI представляє API для програми, і програма стає викликаним і споживає запит. В результаті для вебхуків ролі змінюються; сервер стає клієнтом, а клієнт стає сервером. Кілька програм можуть підписатися на один сервер webhook.

WEBHOOK

Приклад:

Платформа Cisco DNA Center надає веб-хуки, які дозволяють стороннім програмам отримувати дані мережі, коли відбуваються певні події. Ви можете зареєструвати свою програму за допомогою певного URI кінцеве точке REST, якf отримує повідомлення від Cisco DNAC, коли відбувається певна подія.

Наприклад, якщо мережевий пристрій стає недоступним, вебхук Cisco DNAC може надіслати HTTP POST до URI, який ваша програма зареєструвала в Cisco DNAC. Потім ваша програма отримує всі деталі збою в об'єкті JSON з цього HTTP POST, щоб вона могла вжити відповідних заходів. У цьому випадку Cisco DNAC є постачальником вебхуку.

Ви можете створити веб-хук, щоб Cisco Webex Teams сповіщати вас щоразу, коли нові повідомлення публікуються в певній кімнаті. Таким чином, замість того, щоб ваша програма повторювала виклики API Teams, щоб визначити, чи було опубліковано нове повідомлення, вебхук автоматично сповіщає вас про кожне повідомлення. У цьому випадку постачальником вебхуку є Cisco Webex Teams.

WEBHOOK

Сповіщення через Webhook

Щоб отримати сповіщення від постачальника вебхуку, програма має відповідати певним вимогам: Щоб отримувати HTTP-запити POST, програма має працювати постійно. Програма повинна зареєструвати URI на постачальнику вебхуку, щоб постачальник знав, куди надіслати сповіщення, коли відбуваються цільові події. На додаток до цих двох вимог, програма повинна обробляти вхідні сповіщення від сервера webhook. Оскільки до роботи з вебхуками залучаються треті сторони, іноді може бути складно переконатися, що все працює належним чином. Існує багато безкоштовних онлайн-інструментів, які гарантують, що ваша програма може отримувати сповіщення від вебхука. Багато з них також можуть надати вам попередній перегляд вмісту, наданого в сповіщенні вебхуку. Ці інструменти можна знайти за допомогою пошуку «тестер веб-хуку» і можуть бути корисними під час розробки та впровадження вашої програми.

ВИПРАВЛЕННЯ ПОМИЛОК У REST API ЗАПИТАХ

Відсутня відповідь і HTTP Status Code від API server

Почнемо з простого запитання: Чи правда, що кожен запит, надісланий на сервер REST API, повертатиме відповідь із кодом стану? Відповідь хибна. Хоча можна очікувати завжди отримувати код статусу HTTP, такий як 2xx, 4xx і 5xx, є багато випадків, коли сервер API не може бути досягнутий або не відповідає. У таких випадках ви не отримаєте код статусу HTTP, оскільки сервер API не може надіслати відповідь. Хоча сервер, що не відповідає, може бути великою проблемою, першопричину невідповідності можна легко визначити. Зазвичай ви можете визначити, що пішло не так, за повідомленнями про помилки, отриманими в результаті запиту. Давайте розглянемо деякі поради щодо усунення несправностей, які допоможуть вам налагодити, чому сервер API не надіслав відповідь і код статусу HTTP, а також деякі потенційні способи вирішення проблеми.

Помилка на боці клієнта

Перше, що потрібно перевірити, це чи є у вас помилка на стороні клієнта; саме тут ви маєте більше контролю щодо вирішення проблеми.

Чи є помилка від користувача?

Деякі запитання, які потрібно задати, щоб ізолювати помилки від користувача, включають: Чи правильний URI? Чи працював цей запит раніше? Під час першого використання REST API часто трапляється помилковий введення URI. Перевірте довідковий посібник API для цього конкретного API, щоб переконатися, що URI запиту правильний.

ВИПРАВЛЕННЯ ПОМИЛОК У REST API ЗАПИТАХ

Приклад неправильного URI example

Щоб перевірити недійсну умову URI, запустіть такий сценарій, як цей, який просто робить запит до URI, у якому відсутня схема. Ви можете створити файл Python або запустити його безпосередньо в інтерпретаторі Python.

```
import requests
uri = "sandboxdnac.cisco.com/dna/intent/api/v1/network-device"
resp = requests.get(uri, verify = False)
```

Traceback видасть наступну відповідь:

```
....requests.exceptions.MissingSchema: Invalid URL 'sandboxdnac.cisco.com/dna/intent/api/v1
/network-device': No schema supplied. Perhaps you meant http://sandboxdnac.cisco.com/dna/intent
/api/v1/network-device?....
```

Приклад неправильного domain name

Щоб перевірити неправильну умову доменного імені, запустіть такий сценарій, як цей, який просто робить запит до URI з неправильним ім’ям домену.

```
import requests
url = "https://sandboxdnac123.cisco.com/dna/intent/api/v1/network-device"
resp = requests.get(url, verify = False)
```

ВИПРАВЛЕННЯ ПОМИЛОК У REST API ЗАПИТАХ

traceback видасть наступну відповідь:

```
....requests.exceptions.ConnectionError: HTTPSConnectionPool(host='sandboxdnac123.cisco.com',
port=443): Max retries exceeded with url: /dna/intent/api/v1/network-device (Caused by
NewConnectionError('<urllib3.connection.VerifiedHTTPSConnection object at 0x109541080>: Failed
to establish a new connection: [Errno 8] nodename nor servname provided, or not known'))....
```

Чи є проблема з підключенням?

Якщо URI правильний, він все ще може бути недоступним. Задайте ці запитання:

- Чи є проблеми з проксі-сервером, брандмауером або VPN?
- Чи є помилка SSL?

Invalid certificate example

Ця проблема може виникнути лише у випадку, якщо URI REST API використовує захищене з'єднання (HTTPS).Коли схема URI – HTTPS, з'єднання виконує SSL-рукостискання між клієнтом і сервером для їхньої взаємної автентифікації. Це рукостискання має пройти успішно, перш ніж запит REST API буде надісланий на сервер API.

Якщо використовувати бібліотеку requests у Python для виконання запиту REST API і SSL-рукостискання не вдається, у трасуванні помилки (traceback) буде міститися requests.exceptions.SSLError.

ВИПРАВЛЕННЯ ПОМИЛОК У REST API ЗАПИТАХ

Наприклад, якщо SSL handshake не вдається через помилку перевірки сертифіката, то traceback видасть наступне:

```
requests.exceptions.SSLError: HTTPSConnectionPool(host='xxxx', port=443): Max retries exceeded
with url: /dna/intent/api/v1/network-device (Caused by SSLError(SSLCertVerificationError(1,
'[SSL: CERTIFICATE_VERIFY_FAILED] certificate verify failed: self signed certificate
(_ssl.c:1108)')))
```

У цій ситуації необхідно виправити недійсний сертифікат. Але якщо ви працюєте в лабораторному середовищі, де сертифікати ще не дійсні, ви можете вимкнути параметр перевірки сертифіката.

Щоб вимкнути його для бібліотеки requests в Python, додайте параметр verify до запиту.

```
resp = requests.get(url, verify = False)
```

Вирішення: помилки клієнта зазвичай легко виправити, особливо коли повідомлення про помилки в зворотному відстеженні вказують, що може бути не так і можливе рішення. Уважно проаналізуйте логи на предмет першопричини.

ВИПРАВЛЕННЯ ПОМИЛОК У REST API ЗАПИТАХ

Помилка на боці сервера

Після того, як ви переконаєтеся, що немає жодних помилок на стороні клієнта, наступне, що потрібно перевірити, це помилки на стороні сервера

Чи правює взагалі API server?

Найочевидніший момент для початку — переконатися, що сам сервер API функціонує належним чином.

Є кілька речей, які потрібно перевірити:

- Чи ввімкнено живлення?
- Чи немає проблем з кабелем?
- Чи не змінилося доменне ім'я?
- Мережа взагалі працює?

ВИПРАВЛЕННЯ ПОМИЛОК У REST API ЗАПИТАХ

Щоб перевірити, чи доступна IP-адреса, запустіть такий сценарій, як цей, який просто робить запит до URL-адреси та чекає відповіді. **Попередження**: очікуйте велику затримку під час запуску цього сценарію.

```
import requests
url = "https://209.165.209.225/dna/intent/api/v1/network-device"
resp = requests.get(url, verify = False)
```

Примітка: IP-адреса в цьому сценарії є фіктивною, але сценарій дає той самий результат, що й сервер, який недоступний. Якщо сервер API не функціонує, ви отримаєте довге мовчання, за яким слідує traceback, яке виглядає так:

```
....requests.exceptions.ConnectionError: HTTPSConnectionPool(host='209.165.209.225', port=443):
Max retries exceeded with url: /dna/intent/api/v1/network-device (Caused by
NewConnectionError('<urllib3.connection.VerifiedHTTPSConnection object at 0x10502fe20>: Failed
to establish a new connection: [Errno 60] Operation timed out'))....
```

ВИПРАВЛЕННЯ ПОМИЛОК У REST API ЗАПИТАХ

Чи є проблеми зі зв'язком між сервером API і клієнтом? Якщо сервер функціонує належним чином, вам потрібно визначити, чи є причина, по якій клієнт не отримує відповідь.

Потрібно дізнатись:

- Чи доступні IP-адреса та доменне ім'я з місця розташування клієнта в мережі?
- Сервер API надсилає відповідь, але клієнт її не отримує?

Щоб перевірити цю умову, скористайтеся інструментом захоплення мережі, щоб побачити, чи не втрачається відповідь від сервера API під час зв'язку між сервером API і клієнтом. Якщо у вас є доступ, перегляньте журнали сервера API, щоб визначити, чи було отримано запит, чи було його оброблено та чи надіслано відповідь.

Рішення: проблеми на стороні сервера не можна вирішити на стороні клієнта API. Необхідно звернутися до адміністратора API-сервера, щоб вирішити цю проблему.

ІНТЕРПРЕТАЦІЯ КОДІВ СТАТУСУ

Оскільки REST API засновані на http, вони також використовують коди статусу http для сповіщення клієнтів API про результати запиту. Якщо не вказано інше, код статусу є частиною стандарту HTTP/1.1 (RFC 7231), що означає, що перша цифра коду стану визначає клас відповіді. (Останні дві цифри не мають ролі класу чи категоризації, але допомагають розрізняти результати.)

Існує 5 таких категорій:

1xx: Informational: Запит отримано, обробка триває.

2xx: Success: Дія була успішно прийнята, зрозуміла та прийнята.

3xx: Redirection: Щоб завершити запит, необхідно вжити додаткових заходів.

4xx: Client Error: Запит містить неправильний синтаксис або не може бути виконаний.

5xx: Server Error: Сервер не зміг виконати начебто дійсний запит.

Зазвичай ми бачимо 2xx, 4xx і 5xx з сервера REST API. Зазвичай ви можете знайти першопричину помилки, коли зрозумієте значення кодів відповідей. Іноді сервер API також надає додаткову інформацію в тілі відповіді.

Для всіх запитів, які мають код статусу повернення, виконайте такі **дії, щоб усунути помилки**:

Крок 1. Перевірте код повернення. Це може допомогти вивести код повернення у вашому сценарії на етапі розробки.

Крок 2: Перевірте тіло відповіді. Вивести тіло відповіді під час розробки; більшість часу ви можете знайти, що пішло не так, у повідомленні-відповіді, надісланому разом із кодом статусу.

Крок 3. Якщо ви не можете вирішити проблему, використовуючи наведені вище два кроки, скористайтеся посиланням на код статусу, щоб зрозуміти визначення коду стану.

ІНТЕРПРЕТАЦІЯ КОДІВ СТАТУСУ

2xx і 4xx Status Codes

Розглянемо ці коди більш детально:

2xx - Success

Коли клієнт отримує код відповіді 2xx, це означає, що запит клієнта успішно прийнято, зрозуміло та прийнято. Однак ви завжди повинні перевіряти, що відповідь вказує на успіх правильної дії і що сценарій робить те, що, на вашу думку, має.

4xx - Client side error

Відповідь 4xx означає, що помилка на стороні клієнта. Деякі сервери можуть містити об'єкт, що містить пояснення помилки. Якщо ні, ось деякі загальні вказівки щодо усунення поширених помилок 4xx:

400 - Bad Request

Запит не може бути зрозумілий сервером через неправильний синтаксис. Перевірте синтаксис свого API.

Однією з причин помилки 400 Bad Request є сам ресурс. Двічі перевірте кінцеву точку та ресурс, який ви викликаєте, чи не вписали ви один із ресурсів з помилкою чи забули **«s»**, щоб зробити його множиною, наприклад ***/device*** а не ***/devices*** або ***/interface*** замість ***/interfaces***? Чи правильно сформований та повний URI?

Іншою причиною помилки 400 Bad Request може бути проблема синтаксису в об'єкті JSON, який представляє ваш запит POST.

ІНТЕРПРЕТАЦІЯ КОДІВ СТАТУСУ

Наприклад:

```
import requests
url = "http://myservice/api/v1/resources/house/rooms/id/"
resp = requests.get(url,auth=("person2","super"),verify = False)
print (resp.status_code)
print (resp.text)
```

Чи можете ви здогадатися, що не так, просто прочитавши код?

Цей приклад повертає код статусу 400. Сторона сервера також повідомляє вам «Поле ідентифікатора не надано», оскільки ідентифікатор є обов'язковим для цього запиту API. Щоб переконатися, це потрібно буде переглянути в документації.

```
import requests
url = "http://myservice/api/v1/resources/house/rooms/id/1001001027331"
resp = requests.get(url,auth=("person2","super"),verify = False)
print (resp.status_code)
print (resp.text)
```

ІНТЕРПРЕТАЦІЯ КОДІВ СТАТУСУ

401 - Unauthorized

Це повідомлення про помилку означає, що сервер не міг аутентифікувати запит.

Перевірте свої облікові дані, зокрема ім'я користувача, пароль, ключ API, маркер тощо. Якщо з цими елементами немає проблем, ви можете знову перевірити URI запиту, оскільки сервер може відхилити доступ у разі неправильного URI запиту.

```
import requests
url = "http://myservice/api/v1/resources/house/rooms/all"
resp = requests.get(url,verify = False)
print (resp.status_code)
print (resp.text)
```

Цей приклад повертає код статусу 401, чи можете ви вгадати, що не так, просто прочитавши код? Запит не надає облікові дані. У код слід додати аутентифікацію ***auth=("person1","great")***.

```
import requests
url = "http://myservice/api/v1/resources/house/rooms/all"
resp = requests.get(url,auth=("person1","great"),verify = False)
print (resp.status_code)
print (resp.text)
```

403 - Forbidden

У цьому випадку сервер розпізнає облікові дані аутентифікації, але клієнт не має права виконувати запит. Деякі API, наприклад Cisco DNA Center, мають контроль доступу на основі ролей і потребують ролі суперадміністратора для виконання певних API. Знову ж таки, довідковий

```
import requests
url = "http://myservice/api/v1/resources/house/rooms/id/1"
resp = requests.get(url,auth=("person1","great"),verify = False)
print (resp.status_code)
print (resp.text)
```

Чому цей уявний фрагмент коду не працює? Ми просто чудово використали ім'я користувача ***person1*** і пароль ***greate***, і це спрацювало в останньому прикладі.

Код стану 403 не є проблемою аутентифікації; сервер вірить в ідентичність користувача, просто користувач не має достатньо привілеїв для використання цього конкретного API. Якщо ми використовуємо ***person2/super*** як ім'я користувача/пароль для аутентифікації, це спрацює, оскільки person2 має привілей для виконання цього API. Чи можете ви уявити, як це зробити?

Аутентифікація слід змінити, щоб використовувати ***person2/super*** замість ***person1/great***.

```
import requests
url = "http://myservice/api/v1/resources/house/rooms/id/1"
resp = requests.get(url,auth=("person2","super"),verify = False)
print (resp.status_code)
print (resp.text)
```

ІНТЕРПРЕТАЦІЯ КОДІВ СТАТУСУ

404 - Not Found

Сервер не знайшов нічого, що відповідає URI запиту; перевірте URI запиту, щоб переконатися, що він правильний. Якщо раніше код працював, ви можете ознайомитися з останнім довідником API, оскільки синтаксис API може змінюватися з часом.

Розглянемо API «отримати всі інтерфейси» Cisco DNA Center. У заголовку сказано: «усі інтерфейси», тому ви намагаєтеся використовувати ***api/v1/interfaces***, але отримуєте помилку 404, оскільки запит API насправді є ***api/v1/interface***.

```
import requests
url = "http://myservice/api/v1/resources/house/room/all"
resp = requests.get(url,auth=("person1","great"),verify = False)
print (resp.status_code)
print (resp.text)
```

ІНТЕРПРЕТАЦІЯ КОДІВ СТАТУСУ

Чому цей сценарій повернув код статусу 404, і чи можете ви здогадатися, як це виправити?

ІНТЕРПРЕТАЦІЯ КОДІВ СТАТУСУ

Чому цей сценарій повернув код статусу 404, і чи можете ви здогадатися, як це виправити?

Фактичним URI є ***/api/v1/resources/house/rooms/all***. Немає ресурсу ***room***, тому сервер не зміг знайти ресурс, що відповідає URI запиту.

```
import requests
url = "http://myservice/api/v1/resources/house/rooms/all"
resp = requests.get(url,auth=("person1","great"),verify = False)
print (resp.status_code)
print (resp.text)
```

ІНТЕРПРЕТАЦІЯ КОДІВ СТАТУСУ

405 - Method Not Allowed

У цьому випадку запит було розпізнано сервером, але метод, зазначений у запиті, був відхилений сервером. Ви можете переглянути довідковий посібник API, щоб побачити, які методи очікує сервер. Відповідь від сервера також може включати заголовок Allow, що містить список дійсних методів для запитуваного ресурсу.

Наприклад, якщо ви помилково використовуєте метод POST для API, який очікує метод GET, ви отримаєте помилку 405.

```
import requests
url = "http://myservice/api/v1/resources/house/rooms/id/1"
resp = requests.post(url,auth=("person2","super"),verify = False)
print (resp.status_code)
print (resp.text)
```

ІНТЕРПРЕТАЦІЯ КОДІВ СТАТУСУ

406 - Not Acceptable

Ця помилка вказує на те, що цільовий ресурс не має поточного представлення, яке було б прийнятним для клієнта. Сервер має дані, але не може представити їх за допомогою жодного з параметрів, перелічених у заголовках Accept- клієнта.

Наприклад, клієнт запитує зображення SVG: Прийняти: image/svg+xml

```
import requests
headers = {'Accept': 'image/svg+xml'}
url = "http://myservice/api/v1/resources/house/rooms/id/1"
resp = requests.get(url,headers=headers,auth=("person2","super"),verify = False)
print (resp.status_code)
print (resp.text)
```

У цьому випадку сервер не підтримує запитуваний ресурс, image/svg+xml, тому він відповідає помилкою 406.

ІНТЕРПРЕТАЦІЯ КОДІВ СТАТУСУ

407 - Proxy Authentication Required

Цей код схожий на 401 (Неавторизований), але вказує на те, що клієнт повинен спочатку аутентифікуватися за допомогою проксі-сервера. У цьому сценарії між клієнтом і сервером є проксі-сервер, а код відповіді 407 вказує, що клієнту потрібно спочатку пройти автентифікацію за допомогою проксі-сервера.

409 - Не вдалося виконати запит через конфлікт із поточним станом цільового ресурсу .

Наприклад, конфлікт редагування, коли ресурс редагується кількома користувачами, спричинить помилку 409. Повторна спроба запиту пізніше може бути успішною, якщо конфлікт вирішено сервером.

415 - Unsupported Media Type

У цьому випадку клієнт надіслав тіло запиту у форматі, який сервер не підтримує. Наприклад, якщо клієнт надсилає XML на сервер, який приймає лише JSON, сервер поверне помилку 415.

```
import requests
headers = {"content-type":"application/xml"}
url = "http://myservice/api/v1/resources/house/rooms/id/1"
resp = requests.get(url,headers=headers,auth=("person2","super"),verify = False)
print (resp.status_code)
print (resp.text)
```

ІНТЕРПРЕТАЦІЯ КОДІВ СТАТУСУ

З повідомлення про помилку ви можете здогадатися, що може виправити код?
Пропуск заголовка або додавання заголовка **{"content-type":"application/json"}** буде працювати.

```
import requests
headers = {"content-type":"application/json"}

url = "http://myservice/api/v1/resources/house/rooms/id/1"
resp = requests.get(url,headers=headers,auth=("person2","super"),verify = False)
print (resp.status_code)
print (resp.text)
```

Це лише найпоширеніші коди відповіді 4xx. Якщо ви зустрінете інші коди відповіді 4xx, ви можете звернутися до RFC 2616, 6.1, Status-Line або RFC 7231, розділ 6, Коди стану відповіді, щоб отримати додаткову інформацію про те, що вони означають.

ІНТЕРПРЕТАЦІЯ КОДІВ СТАТУСУ

5xx Status Codes

5xx server side error

5xx відповідь повідомляє про помилку на боці серверу.

500 - Internal Server Error Ця помилка означає, що сервер зіткнувся з несподіваною умовою, яка завадила йому виконати запит.

501 - Not Implemented Ця помилка означає, що сервер не підтримує функції, необхідні для виконання цього запиту. Наприклад, сервер відповість кодом 501, якщо він не розпізнає метод запиту і тому не може підтримувати його для будь-якого ресурсу.

502 - Bad Gateway Ця помилка означає, що сервер, виконуючи роль шлюзу або проксі-сервера, отримав недійсну відповідь від вхідного сервера, до якого він звертався, намагаючись виконати запит.

503 - Service Unavailable Цей код вказує, що сервер наразі не може обробити запит через тимчасове перевантаження або планове обслуговування, яке, ймовірно, буде вирішено після затримки.

504 - Gateway Timeout Ця помилка означає, що сервер, виконуючи роль шлюзу або проксі, не отримав своєчасної відповіді від вищестоящого сервера, до якого йому потрібно було отримати доступ, щоб виконати запит.

Якщо ви отримуєте помилку 500 або 501, перевірте довідковий посібник API, щоб переконатися, що запит дійсний. Щодо інших помилок 5xx зверніться до адміністратора сервера API, щоб вирішити проблему. Перш ніж почати вирішувати проблеми з API, дуже важливо мати під рукою довідковий посібник із API та коди стану. Я кщо ви не можете отримати код відповіді, ви, ймовірно, зможете визначити причину з журналу помилок у вашому сценарії. Якщо ви можете отримати код відповіді, ви зможете визначити основну причину помилки, зокрема, чи є помилка на стороні клієнта чи сервера, розуміючи код відповіді.

Q&A